

INTERNSHIP PROJECT REPORT TASK-3

Title

Simulating and Responding to a Multi-Stage APT Attack Using Open-Source SOC Tools
Phase 1: Attack Simulation (Red Team) • Gain initial access through phishing or exploiting an unpatched vulnerability. • Establish persistence and escalate privileges using scheduled tasks or PowerShell abuse. • Move laterally across the network using PsExec or Pass-the-Hash techniques. • Exfiltrate data using covert DNS tunneling or encrypted HTTP requests. Phase 2: Attack Detection & Investigation (Blue Team) • Detect suspicious activities using Security Onion and Wazuh alerts. • Hunt for IoCs using Kibana queries and correlate logs from multiple sources. • Investigate malicious files, URLs, and IPs using TheHive & Cortex. • Reconstruct the attack timeline and write a forensic report. Phase 3: Threat Intelligence & Mitigation • Extract IoCs and share them via MISP. • Write YARA and Sigma rules to prevent similar attacks. • Implement a mitigation plan to block attack vectors. • Present an incident response report detailing security gaps and remediation

1. Introduction

As part of my cybersecurity internship, I was assigned the task of **Simulating and Responding to a Multi-Stage APT Attack Using Open-Source SOC Tools**. The main objective of this project was to understand how an Advanced Persistent Threat (APT) attack can take place in different stages and how security teams can detect, investigate and respond to suspicious activities.

In this project, I performed attack simulation activities in a controlled environment and monitored the generated events using **Elastic Cloud, Elastic Agent, Elastic Defend, and Kibana**. The activities included user account creation, privilege escalation, scheduled task creation for persistence, PowerShell execution and process enumeration. The generated endpoint events were then collected and analyzed using Kibana queries.

2. Background

Advanced Persistent Threat (APT) attacks are complex cyber attacks that usually happen in multiple stages. Instead of attacking a system only once, an attacker may try to gain access, maintain access for a longer period, increase privileges, move to other systems and steal important data.

In a Security Operations Center (SOC), the Red Team and Blue Team have different roles. The Red Team simulates attacker techniques to test security systems, while the Blue Team monitors and investigates suspicious activities. Tools such as SIEM platforms, endpoint security tools, threat intelligence platforms and detection rules can help security analysts identify suspicious activities and understand what happened during an attack.

3. Learning Objectives

The main objective of this project was to gain practical knowledge of how a multi-stage APT attack can be simulated, detected, investigated, and responded to using SOC and security monitoring tools.

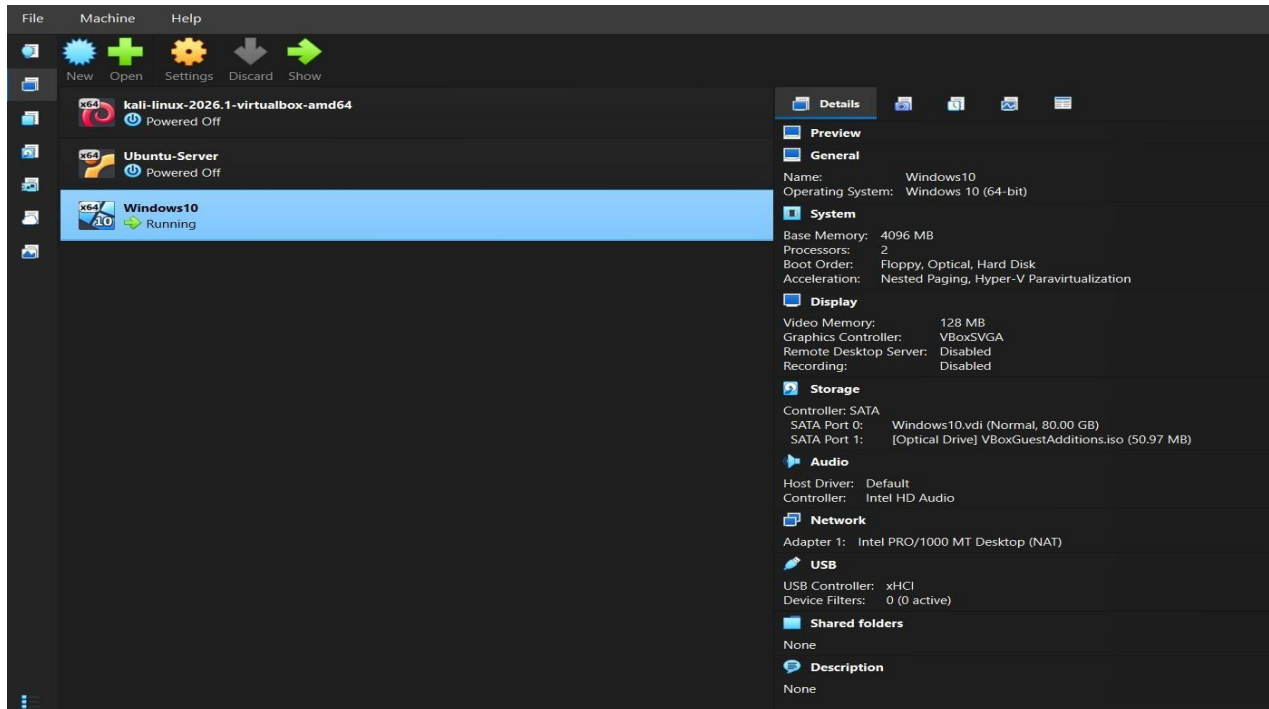
The main objectives of this project were:

- To Understand the different stages of an APT attack.
- Learn basic attack simulation techniques in a controlled environment.
- Understand how endpoint logs and security events are collected.
- To Investigate suspicious activities using Kibana queries.
- To Identify possible Indicators of Compromise (IoCs).
- Understand the role of Red Team, Blue Team, and Purple Team activities.

4. Activities and Tasks Performed

Step 1 - Setting up Windows 10 Virtual Machine

First, I set up a **Windows 10 virtual machine using VirtualBox**. This Windows 10 system was used as the target machine for performing the project activities. After completing the setup, the Windows 10 system was started successfully and prepared for the next stages of the APT attack simulation.



Step 2 - Checking and Installing Basic Tools in Kali Linux

After setting up the Windows 10 and Kali Linux systems, I checked whether the basic tools required for this project were available in Kali Linux. I checked **Metasploit**, **Nmap** and **Python** by using version commands. After checking these tools, I installed some additional basic utilities such as **net-tools**, **curl**, **wget**, **Git**, **zip** and **unzip**. These tools can be useful for network checking, downloading files, managing project files, and running different tasks.

```

kali@kali: ~
Session Actions Edit View Help

zsh: corrupt history file /home/kali/.zsh_history

kali@kali:~$ cat /dev/null > .zsh_history
kali@kali:~$ sudo apt-get update
Framework Version: 6.4.135-dev

kali@kali:~$ sudo apt-get install -y nmap net-tools curl wget git python3 python3-pip zip unzip
nmap version 7.99 (https://nmap.org)
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.8 openssl-3.6.2 libssh2-1.11.1 libpcap-1.10.6 nmap-libndm-1.10.0 ipv6
Compiled without:
Available nsock engines: epoll poll select

kali@kali:~$ python3 --version
Python 3.13.14

kali@kali:~$ sudo apt install -y nmap net-tools curl wget git python3 python3-pip zip unzip
[sudo] password for kali:
nmap is already the newest version (7.99dfsg-1kali1).
net-tools is already the newest version (2.10-2).
curl is already the newest version (8.20.0-5).
wget is already the newest version (1.25.0-2).
git is already the newest version (1:2.53.0-1).
python3 is already the newest version (3.13.9-3+b1).
python3-pip is already the newest version (26.1.2dfsg-1).
zip is already the newest version (3.0-15).
unzip is already the newest version (6.0-29).
The following packages were automatically installed and are no longer required:
  libbsd0 libbz2-1.0 libedit2 libffi8 libgdbm6 libgdbm-compat4 libldap2 liblzma5 libncurses6 libnss3 libp11-kit0 libpython3.13-minimal libpython3.13-stdlib libpython3.13-venv libpython3.13_venv libsqlite3-0 libssl3t64 libstdc++6 libsystemd0 libtasn1-6 libtinfo6 libunistring2 libuuid1 libwrap0 python3-pluginbase python3-tz
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 34

kali@kali:~$

```

Step 3 - Installing Required Tools in Kali Linux

Next, I installed the required security tools in **Kali Linux**. These tools were used for performing different security testing and attack simulation activities in this project.

The tools installed were:

- **Metasploit Framework** - used for security testing and attack simulation.
- **SMBClient** - used to access and test shared files and folders over a network.
- **Enum4linux** - used to collect basic information from Windows systems and network services.
- **Impacket Scripts** - used for testing different Windows and network-related activities.
- **DNSUtils** - used for checking and testing DNS-related activities.

```

kali@kali:~$ sudo apt install -y metasploit-framework
[sudo] password for kali:
metasploit-framework is already the newest version (6.4.135-0kali1).
metasploit-framework set to manually installed.
The following packages were automatically installed and are no longer required:
  enchant-2 libhaci194-0 libio-pty-perl libmbedcrypto16 libpostal-data libraw23t64 libssmduf3 libsvtavcodec libunibreak6 libxmlsec1-1 libxring3 python3-pluginbase python3-tz
  libhaci2824722 libgdal18 libipc-run-perl libopenjph27 libpostal1 libssl2-classic libssodm23 libteamdctl0 libvidstab1.1 libxmlsec1-openssl1 libxring3 postgresql-common-dev python3-pluginbase python3-tz
  libhaci2824722 libgdal18 libipc-run-perl libopenjph27 libpostal1 libssl2-classic libssodm23 libteamdctl0 libvidstab1.1 libxmlsec1-openssl1 libxring3 postgresql-common-dev python3-pluginbase python3-tz
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 24

kali@kali:~$ sudo apt install -y smbclient
[sudo] password for kali:
smbclient is already the newest version (2:4.24.3+dfsg-1).
smbclient set to manually installed.
The following packages were automatically installed and are no longer required:
  enchant-2 libhaci194-0 libio-pty-perl libmbedcrypto16 libpostal-data libraw23t64 libssmduf3 libsvtavcodec libunibreak6 libxmlsec1-1 libxring3 python3-pluginbase python3-tz
  libhaci2824722 libgdal18 libipc-run-perl libopenjph27 libpostal1 libssl2-classic libssodm23 libteamdctl0 libvidstab1.1 libxmlsec1-openssl1 libxring3 postgresql-common-dev python3-pluginbase python3-tz
  libhaci2824722 libgdal18 libipc-run-perl libopenjph27 libpostal1 libssl2-classic libssodm23 libteamdctl0 libvidstab1.1 libxmlsec1-openssl1 libxring3 postgresql-common-dev python3-pluginbase python3-tz
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 24

kali@kali:~$ sudo apt install -y enumlinux
[sudo] password for kali:
enumlinux is already the newest version (0.9.1-0kali2).
enumlinux set to manually installed.
The following packages were automatically installed and are no longer required:
  enchant-2 libhaci194-0 libio-pty-perl libmbedcrypto16 libpostal-data libraw23t64 libssmduf3 libsvtavcodec libunibreak6 libxmlsec1-1 libxring3 python3-pluginbase python3-tz
  libhaci2824722 libgdal18 libipc-run-perl libopenjph27 libpostal1 libssl2-classic libssodm23 libteamdctl0 libvidstab1.1 libxmlsec1-openssl1 libxring3 postgresql-common-dev python3-pluginbase python3-tz
  libhaci2824722 libgdal18 libipc-run-perl libopenjph27 libpostal1 libssl2-classic libssodm23 libteamdctl0 libvidstab1.1 libxmlsec1-openssl1 libxring3 postgresql-common-dev python3-pluginbase python3-tz
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 24

kali@kali:~$ sudo apt install -y impactet-scripts
[sudo] password for kali:
impactet-scripts is already the newest version (1.10).
impactet-scripts set to manually installed.
The following packages were automatically installed and are no longer required:
  enchant-2 libhaci194-0 libio-pty-perl libmbedcrypto16 libpostal-data libraw23t64 libssmduf3 libsvtavcodec libunibreak6 libxmlsec1-1 libxring3 python3-pluginbase python3-tz
  libhaci2824722 libgdal18 libipc-run-perl libopenjph27 libpostal1 libssl2-classic libssodm23 libteamdctl0 libvidstab1.1 libxmlsec1-openssl1 libxring3 postgresql-common-dev python3-pluginbase python3-tz
  libhaci2824722 libgdal18 libipc-run-perl libopenjph27 libpostal1 libssl2-classic libssodm23 libteamdctl0 libvidstab1.1 libxmlsec1-openssl1 libxring3 postgresql-common-dev python3-pluginbase python3-tz
Use 'sudo apt autoremove' to remove them.

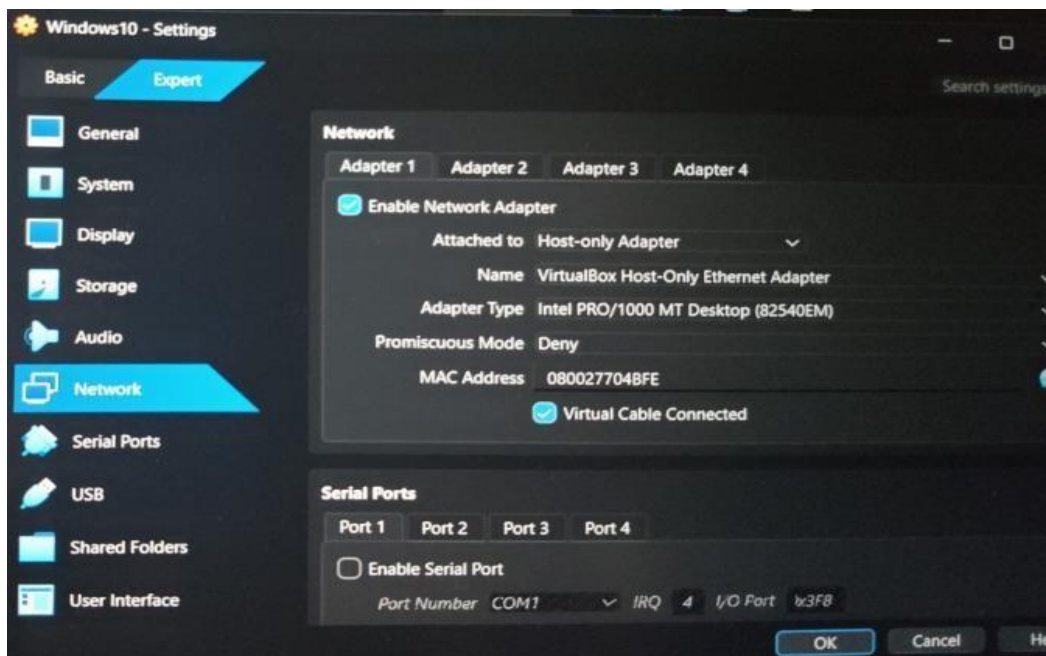
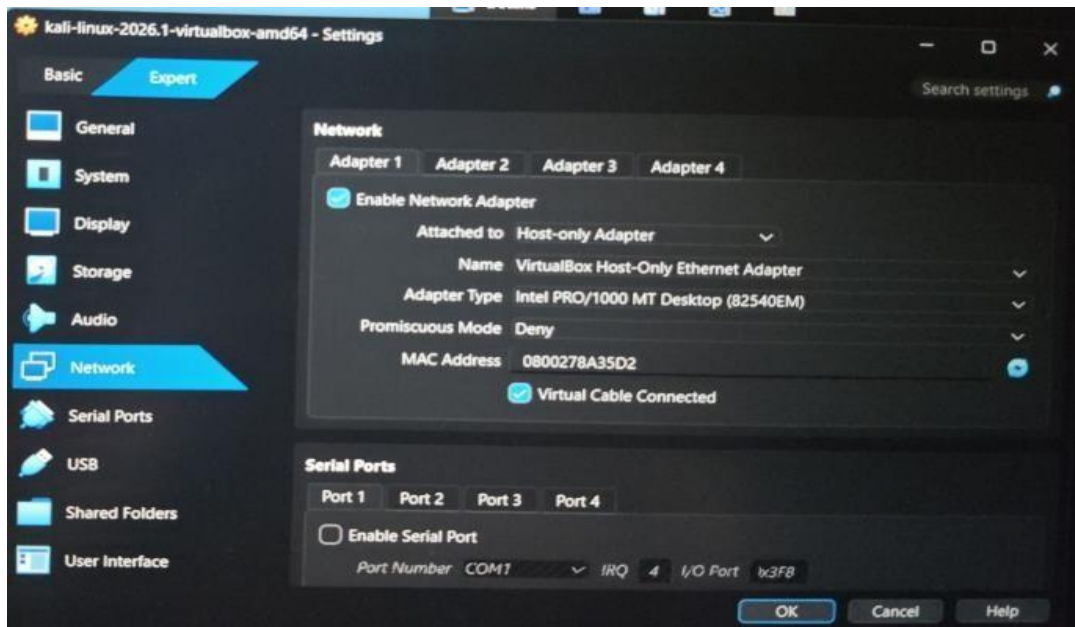
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 24

kali@kali:~$ sudo apt install -y dnsutils
[sudo] password for kali:

```

Step 4 - Configuring Network Connection between Virtual Machines

In this step, I configured the network settings for both the **Kali Linux** and **Windows 10** virtual machines. I selected the **Host-Only Adapter** option in VirtualBox for both machines. This allowed the two virtual machines to communicate with each other in an isolated network. Kali Linux was used for performing the attack simulation, while Windows 10 was used as the target machine.



Step 5 - Checking the Network Connection between Virtual Machines

After setting the network, I checked whether the Windows 10 and Kali Linux machines could communicate with each other. I used the **ipconfig** command in Windows to check its IP address. In Kali Linux, I used the **ip addr** command to view the network details and IP address. After that, I used the **ping** command to test the connection between the two machines. The ping replies showed that Windows 10 and Kali Linux were connected and communicating with each other properly. This network connection was then used for the next steps of the attack simulation.

```
cmd Select Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::48a8:ddd1:2f94:f12c%5
    IPv4 Address. . . . . : 192.168.56.102
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

C:\Windows\system32>
```

```
C:\Windows\system32>ping 192.168.56.101

Pinging 192.168.56.101 with 32 bytes of data:
Reply from 192.168.56.101: bytes=32 time=1ms TTL=64
Reply from 192.168.56.101: bytes=32 time<1ms TTL=64
Reply from 192.168.56.101: bytes=32 time=3ms TTL=64
Reply from 192.168.56.101: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.56.101:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 3ms, Average = 1ms

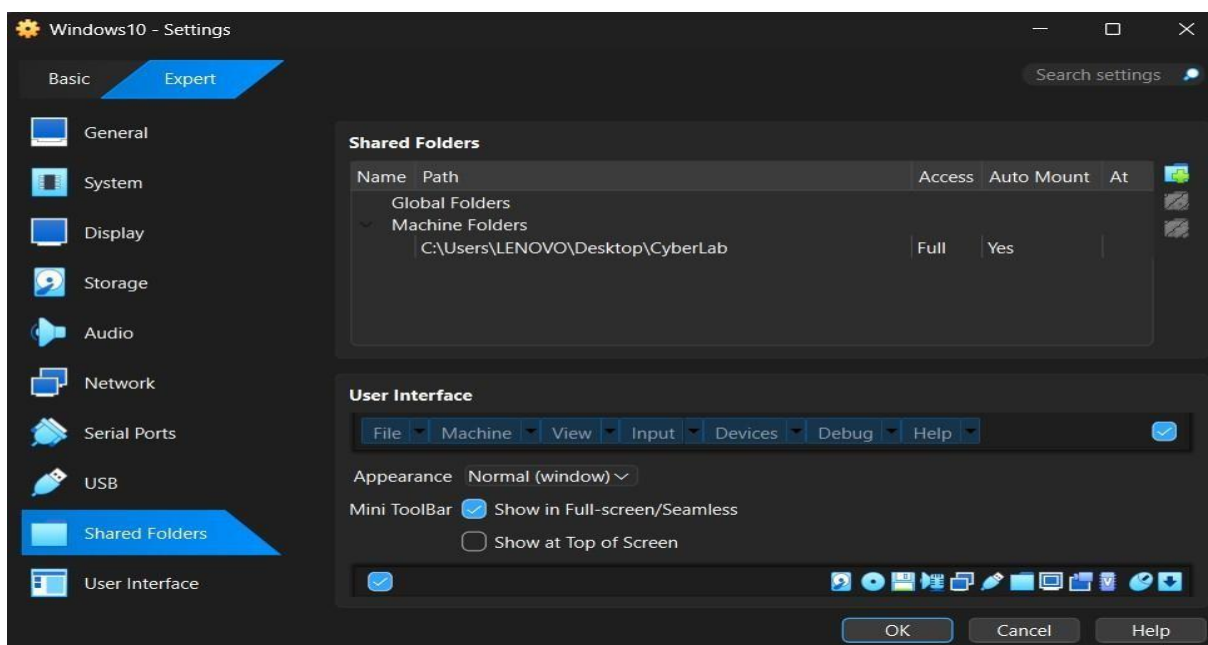
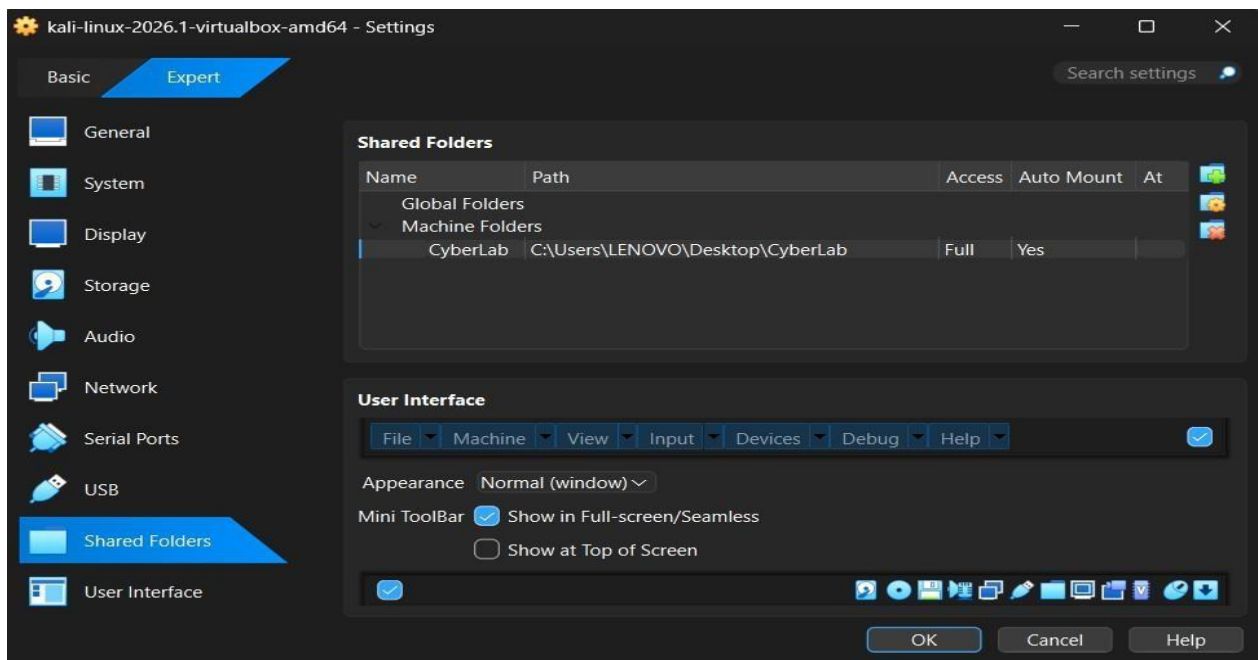
C:\Windows\system32>
```

```
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
kali@kali:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0a:35:42 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0
        valid_lft 572sec preferred_lft 572sec
    inet6 fe80::270a:3542:0000:0000/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: dockero: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 5a:5a:00:02:00:00 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/24 brd 172.17.0.255 scope global dockero
        valid_lft forever preferred_lft forever

kali@kali:~$
```

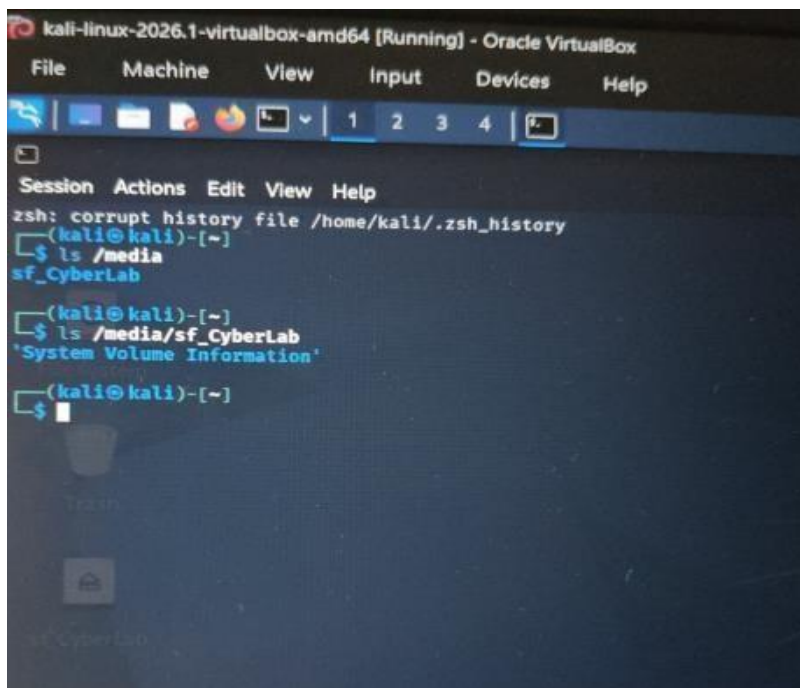
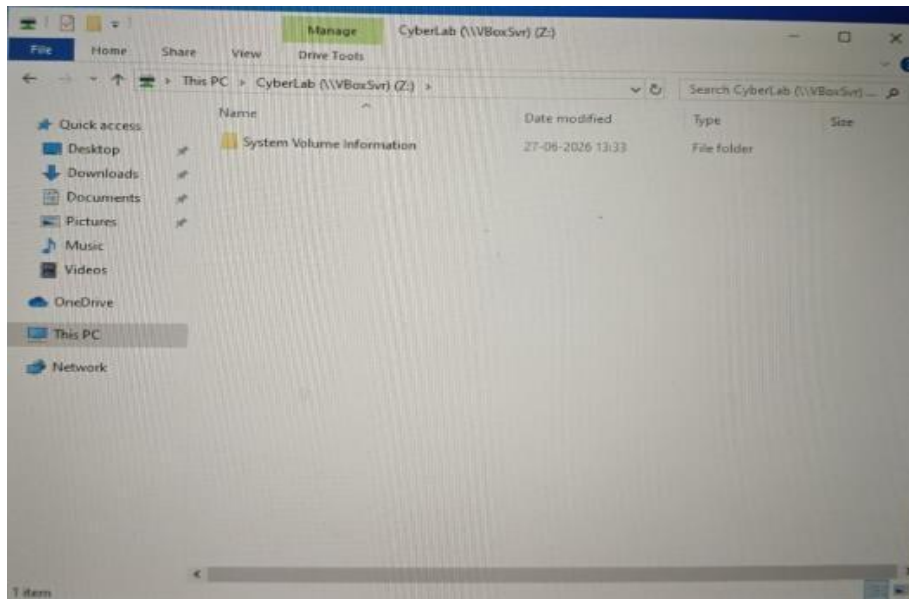

Step 6 - Setup Shared Folder between Virtual Machines

In this step, I created a shared folder named **CyberLab** on the main system. Then, I added the same folder to both the **Kali Linux** and **Windows 10** virtual machines using the **Shared Folders** option in VirtualBox. The **Auto Mount** option was enabled so that the shared folder could be available automatically when the virtual machines started. This shared folder was used to easily transfer files between the main system, Kali Linux and Windows 10 during the project.



Step 7 - Checking the Shared Folder in both Virtual Machines

In this step, I checked whether the **CyberLab** shared folder was working properly in both Windows 10 and Kali Linux. In Windows 10, the shared folder was available as a drive named **CyberLab (Z:)**. In Kali Linux, I used the `ls /media` command to check the shared folder and the **CyberLab** folder was displayed. This confirmed that the shared folder was successfully connected to both virtual machines.



Step 8 - Scan the Windows Machine using NMAP

In this step, I used **Nmap** in Kali Linux to scan the Windows 10 machine with the IP address **192.168.56.102**.

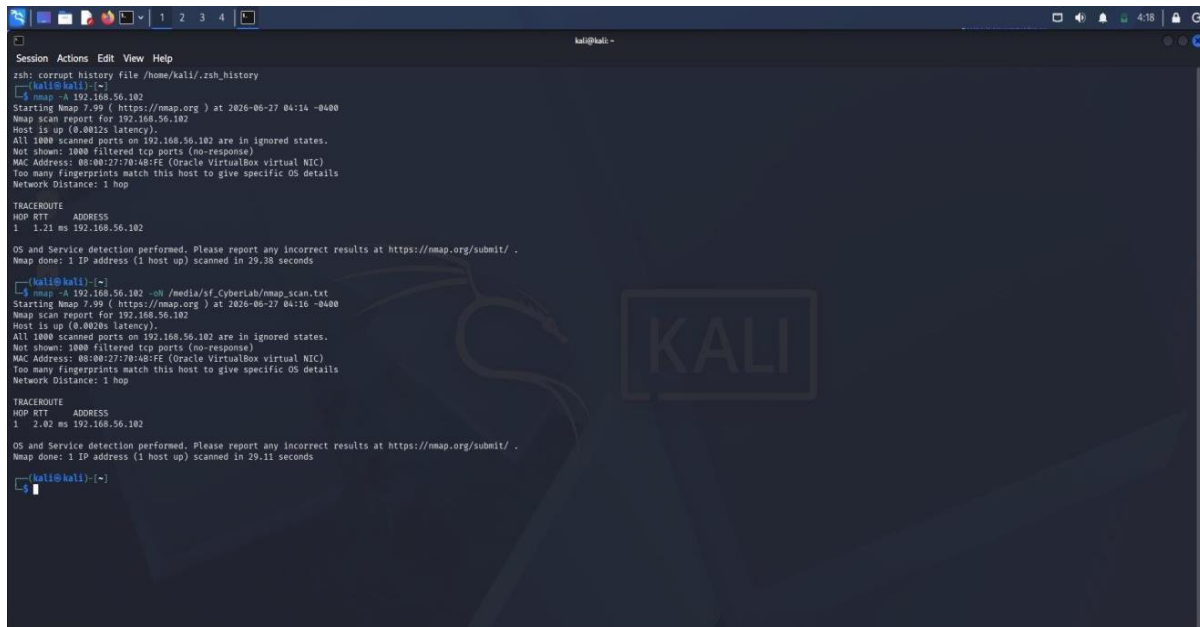
I ran the following command: **nmap -A 192.168.56.102**

The result showed that the Windows 10 machine was **up and reachable** from Kali Linux. This confirmed that the network connection between both virtual machines was working properly. However, the scan showed that the tested TCP ports were filtered or did not respond, so no open ports were found.

After that, I ran the scan again and saved the result using the command:

nmap -A 192.168.56.102 -oN /media/sf_CyberLab/nmap_scan.txt

This saved the Nmap scan output as **nmap_scan.txt** in the **CyberLab** shared folder. This allowed the scan result to be accessed later from both Kali Linux and Windows 10.



```
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
kali@kali:~$ nmap -A 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2020-06-27 04:14 -0400
Nmap scan report for 192.168.56.102
Host is up (0.0021s latency).
All 1000 scanned ports on 192.168.56.102 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 08:00:27:78:48:FE (Oracle VirtualBox virtual NIC)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

TRACEROUTE
HOP RTT ADDRESS
1 1.21 ms 192.168.56.102

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 29.38 seconds

kali@kali:~$ nmap -A 192.168.56.102 -oN /media/sf_CyberLab/nmap_scan.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2020-06-27 04:16 -0400
Nmap scan report for 192.168.56.102
Host is up (0.0028s latency).
All 1000 scanned ports on 192.168.56.102 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 08:00:27:78:48:FE (Oracle VirtualBox virtual NIC)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

TRACEROUTE
HOP RTT ADDRESS
1 2.02 ms 192.168.56.102

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 29.11 seconds

kali@kali:~$
```

Step 9 - Scanning Open Ports and Enumerating the Target

In this step, I performed a detailed scan on the Windows 10 target machine from Kali Linux. First, I used the following Nmap command: **nmap -p 445 192.168.56.102**

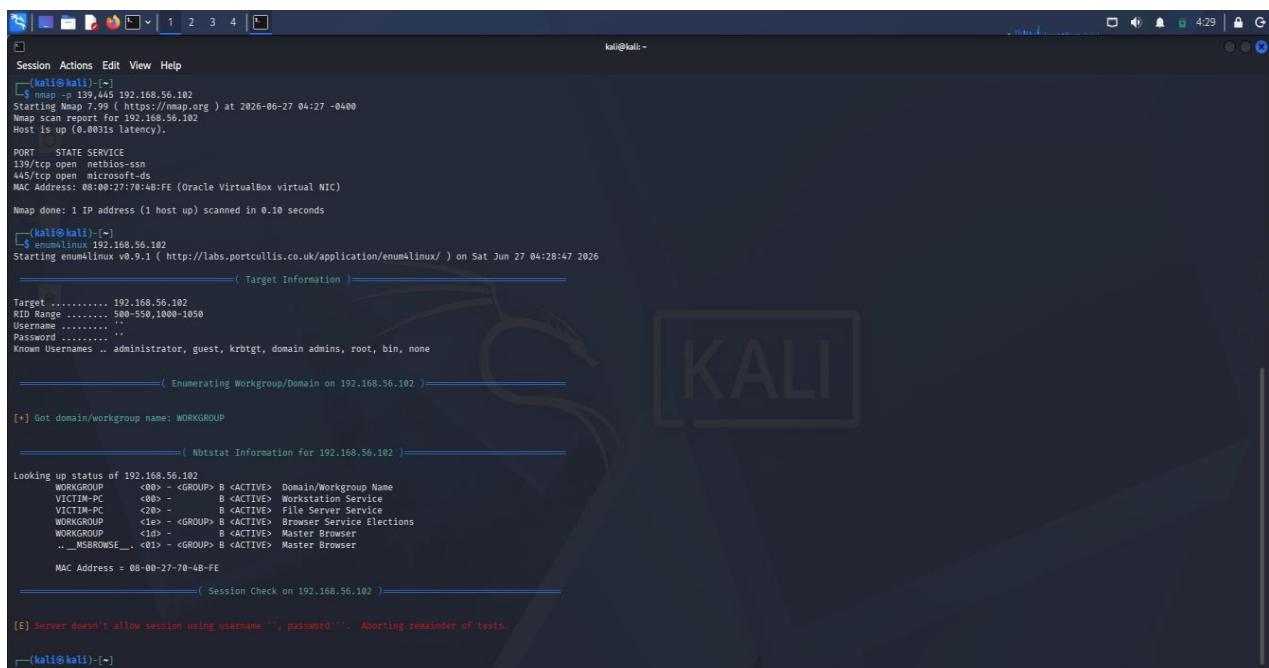
The scan showed that **port 139** and **port 445** were open on the Windows machine. These ports are commonly related to Windows networking and file-sharing services.

After identifying the available Windows services, I used the following command:
enum4linux 192.168.56.102

I used **enum4linux** to collect additional information from the Windows machine. This tool is mainly used to gather details related to Windows and SMB services.

The scan returned information such as the **workgroup name**, which was shown as **WORKGROUP**, and the computer name **VICTIM-PC**. It also displayed information related to the file-sharing services running on the system.

During the enumeration process, the tool later showed that the server did not allow the tested session, so it could not collect further information.



```
kali@kali: ~  
└─( kali@kali )─┘  
Session Actions Edit View Help  
[kali@kali]~  
$ nmap -p 139,445 192.168.56.102  
Starting Nmap 7:99 ( https://nmap.org ) at 2026-06-27 04:27 -0400  
Nmap scan report for 192.168.56.102  
Host is up (0.0031s latency).  
  
PORT      STATE SERVICE  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
MAC Address: 08:00:27:70:4B:FE (Oracle VirtualBox virtual NIC)  
Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds  
[kali@kali]~  
$ enum4linux 192.168.56.102  
Starting enum4linux v0.9.1 ( http://labs.portcullis.co.uk/application/enum4linux/ ) on Sat Jun 27 04:28:47 2026  
  
----- ( Target Information ) -----  
Target ..... 192.168.56.102  
RID Range ..... 500-550,1000-1050  
Username ..... ''  
Password ..... ''  
Known Usernames .. administrator, guest, krbtgt, domain admins, root, bin, none  
  
----- ( Enumerating Workgroup/Domain on 192.168.56.102 ) -----  
[+] Got domain/workgroup name: WORKGROUP  
  
----- ( Nbtstat Information for 192.168.56.102 ) -----  
Looking up status of 192.168.56.102  
WORKGROUP <00> - <GROUP> B <ACTIVE> Domain/Workgroup Name  
VICTIM-PC <00> - B <ACTIVE> Workstation Service  
VICTIM-PC <20> - B <ACTIVE> File Server Service  
WORKGROUP <10> - <GROUP> B <ACTIVE> Browser Service Elections  
WORKGROUP <10> - B <ACTIVE> Master Browser  
.._MSBROWSE... <01> - <GROUP> B <ACTIVE> Master Browser  
MAC Address = 08-00-27-70-4B-FE  
  
----- ( Session Check on 192.168.56.102 ) -----  
[E] Server doesn't allow session using username '', password ''. Aborting remainder of tests.  
[kali@kali]~
```

Step 10 - Checking SMB Access and Additional Ports

In this step, I continued testing the Windows target machine **192.168.56.102** to check whether the SMB file-sharing service could be accessed and to examine additional ports.

First, I used the command: **smbclient -L //192.168.56.102 -U student**

I used **smbclient** to check the shared folders and services available through SMB on the Windows machine. The **-L** option was used to list the available shares, and **-U student** was used to try accessing the service with the **student** username.

The result showed shares such as **ADMIN\$**, **C\$**, **IPC\$** and **Public**. These are related to Windows administration, file sharing, and communication services.

However, the connection attempt was not fully successful and showed an error. This indicated that additional access or correct permissions were required.

Next, I used: **smbclient //192.168.56.102/Public -U student**

I used this command to directly connect to the **Public** shared folder. The purpose was to check whether the shared folder could be accessed using the available username.

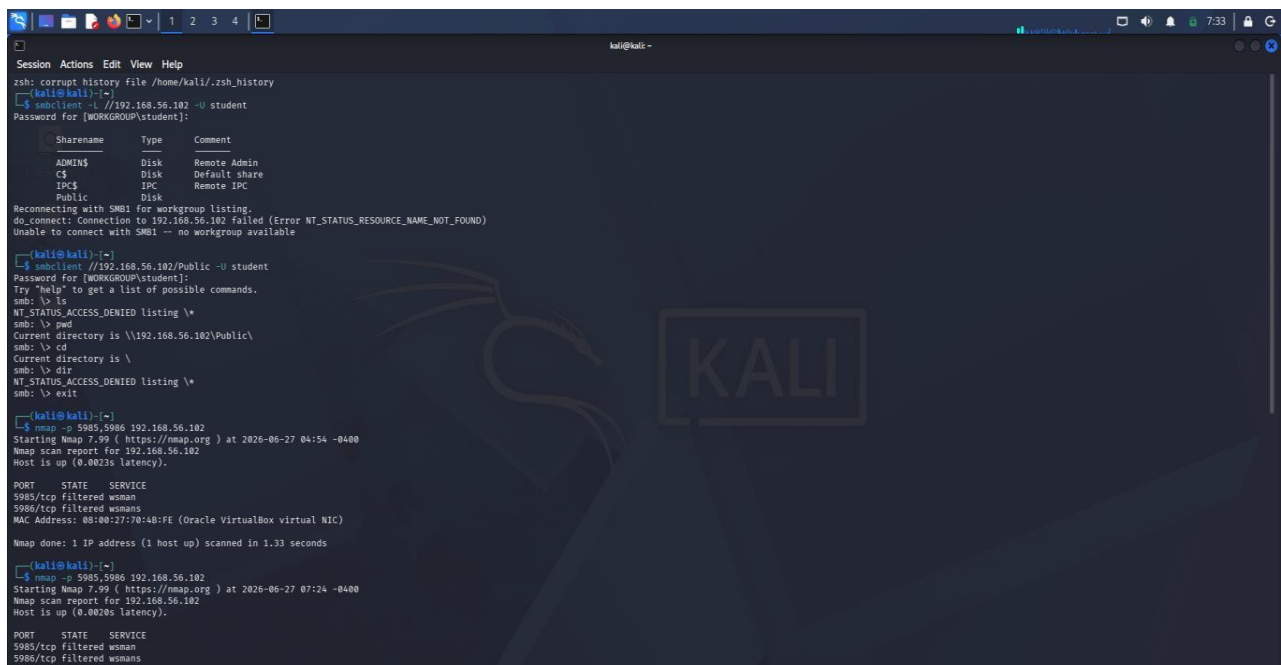
The connection was established, but when I tried to view the contents using the **ls** command, the system returned an **ACCESS_DENIED** message. This showed that the user did not have permission to list or access the files inside the shared folder.

After checking the SMB service, I also scanned ports **5985** and **5986** using Nmap:

nmap -p 5985,5986 192.168.56.102

The scan result showed that port **5985** was filtered, while port **5986** was also not available for direct access.

This means that these remote management services could not be directly accessed from the Kali machine during the test.



```
kali@kali: ~  
Session Actions Edit View Help  
zsh: corrupt history file /home/kali/.zsh_history  
kali@kali:~$ smbclient -L //192.168.56.102 -U student  
Password for [WORKGROUP\student]:  
Sharename      Type            Comment  
-----  
ADMIN$         Disk            Remote Admin  
C$              Disk            Default share  
IPC$           IPC             Remote IPC  
Public          Disk  
Reconnecting with SMB1 for workgroup listing.  
do_connect: Connection to 192.168.56.102 failed (Error NT_STATUS_RESOURCE_NAME_NOT_FOUND)  
Unable to connect with SMB1 -- no workgroup available  
kali@kali:~$ smbclient //192.168.56.102/Public -U student  
Password for [WORKGROUP\student]:  
Try "help" to get a list of possible commands.  
smb: \> ls  
NT_STATUS_ACCESS_DENIED listing \>  
smb: \> pwd  
Current directory is \\192.168.56.102\Public\  
smb: \> cd  
Current directory is \  
smb: \> dir  
NT_STATUS_ACCESS_DENIED listing \  
smb: \> exit  
kali@kali:~$ nmap -p 5985,5986 192.168.56.102  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-27 04:54 -0400  
Nmap scan report for 192.168.56.102  
Host is up (0.0023s latency).  
  
PORT      STATE SERVICE  
5985/tcp  filtered wsman  
5986/tcp  filtered wsman  
MAC Address: 08:00:27:70:4B:FE (Oracle VirtualBox virtual NIC)  
Nmap done: 1 IP address (1 host up) scanned in 1.33 seconds  
kali@kali:~$ nmap -p 5985,5986 192.168.56.102  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-27 07:24 -0400  
Nmap scan report for 192.168.56.102  
Host is up (0.0020s latency).  
  
PORT      STATE SERVICE  
5985/tcp  filtered wsman  
5986/tcp  filtered wsman
```

Step 11 - Enabling and Checking WinRM

In this step, I configured and checked the **Windows Remote Management (WinRM)** service on the Windows 10 machine.

First, I used the following command in Command Prompt: **winrm quickconfig**
I used this command to configure the WinRM service. The command checks whether WinRM is running and sets up the basic configuration required to receive remote management requests. This happened because the current network was set to **Public**, which can restrict some remote connections for security reasons.

Next, I used the command: **sc query WinRM**

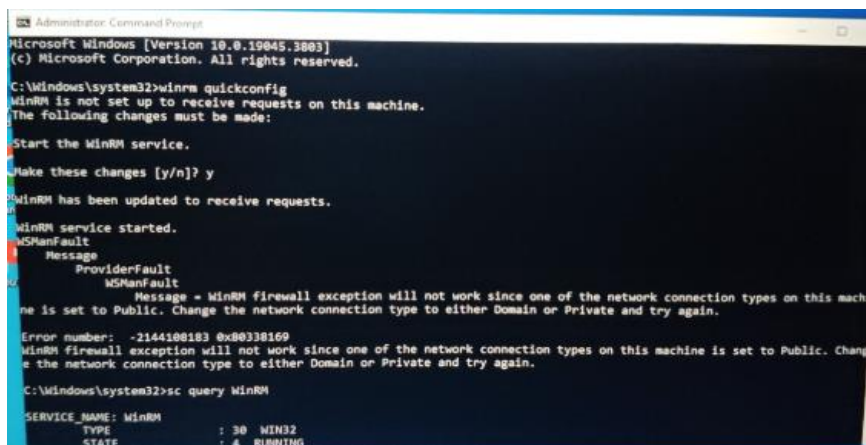
I used this command to check the current status of the WinRM service.

The result showed that the **WinRM service was running**. This confirmed that the remote management service was successfully started on the Windows machine.

After confirming that WinRM was running, I checked the Windows Firewall rule using:

netsh advfirewall firewall show rule name="Windows Remote Management (HTTP-In)"

The output showed that the **Windows Remote Management (HTTP-In)** rule was enabled. It also showed that the rule was associated with **TCP port 5985**, which is used for WinRM communication over HTTP.



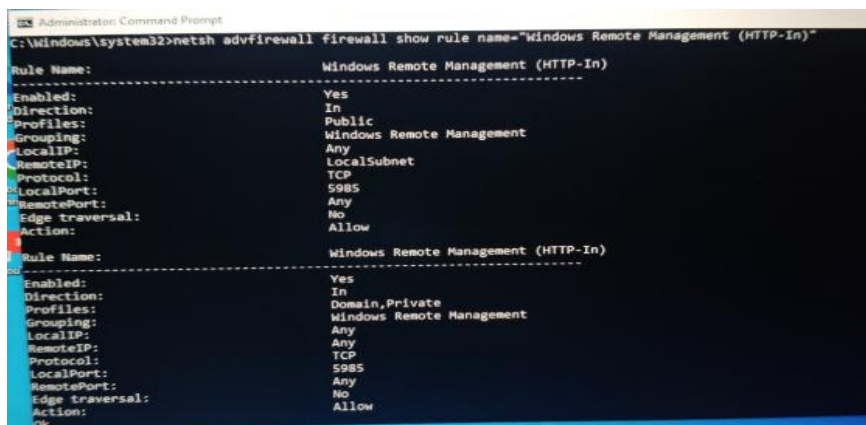
```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>winrm quickconfig
WinRM is not set up to receive requests on this machine.
The following changes must be made:

Start the WinRM service.
Make these changes [y/n]? y
WinRM has been updated to receive requests.

WinRM service started.
WSManFault
  Message
    ProviderFault
      WSManFault
        Message = WinRM firewall exception will not work since one of the network connection types on this machine is set to Public. Change the network connection type to either Domain or Private and try again.
Error number: -2144108183 0x80338169
WinRM firewall exception will not work since one of the network connection types on this machine is set to Public. Change the network connection type to either Domain or Private and try again.

C:\Windows\system32>sc query WinRM
SERVICE_NAME: WinRM
        TYPE               : 30  WIN32
        STATE                : 4   RUNNING
```



```
Administrator: Command Prompt
C:\Windows\system32>netsh advfirewall firewall show rule name="Windows Remote Management (HTTP-In)"

Rule Name:                               Windows Remote Management (HTTP-In)
-----
Enabled:                                   Yes
Direction:                               In
Profiles:                                 Public
Grouping:                                 Windows Remote Management
LocalIP:                                  Any
RemoteIP:                                  LocalSubnet
Protocol:                                  TCP
LocalPort:                                 5985
RemotePort:                                Any
Edge traversal:                             No
Action:                                    Allow

Rule Name:                               Windows Remote Management (HTTP-In)
-----
Enabled:                                   Yes
Direction:                               In
Profiles:                                 Domain,Private
Grouping:                                 Windows Remote Management
LocalIP:                                  Any
RemoteIP:                                  Any
Protocol:                                  TCP
LocalPort:                                 5985
RemotePort:                                Any
Edge traversal:                             No
Action:                                    Allow
Ok.
```

Step 12 - Changing the Network Profile and Configuring WinRM

In this step, I changed the network profile of the Windows 10 machine from **Public** to **Private** and then configured the WinRM service again.

First, I opened Windows PowerShell as Administrator and used the following command:

Get-NetConnectionProfile

I used this command to check the current network configuration of the Windows machine.

Next, I used the following command: **Set-NetConnectionProfile -NetworkCategory Private**

I used this command to change the network category from **Public** to **Private**.

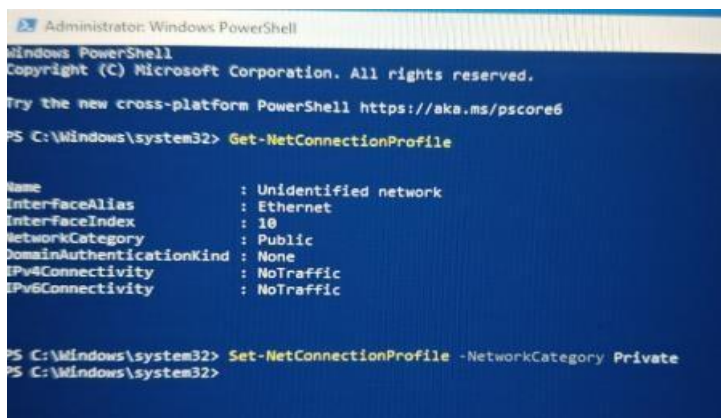
The Private network setting is more suitable for this controlled lab because the Kali Linux and Windows machines were intentionally connected for testing.

Next, I opened Command Prompt as Administrator and used the following command:

winrm quickconfig

I used this command to check the current WinRM configuration and make the required changes for remote management.

After confirming the changes, Windows updated the required settings for remote management.



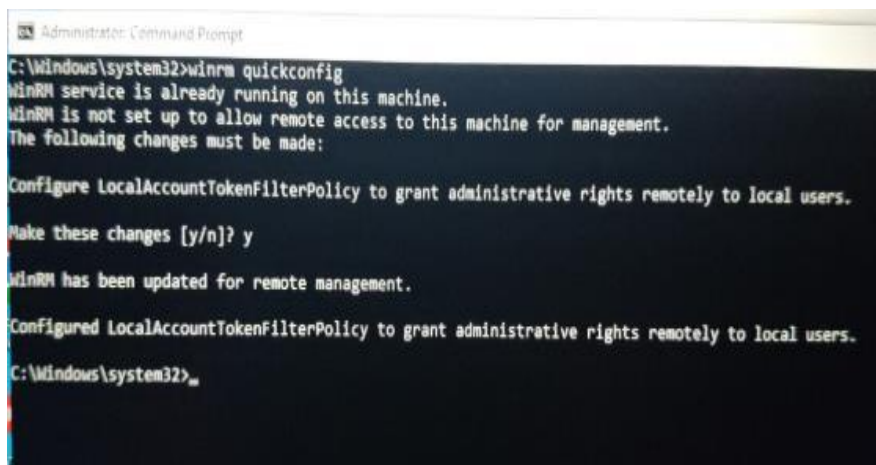
```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> Get-NetConnectionProfile

Name                : Unidentified network
InterfaceAlias       : Ethernet
InterfaceIndex       : 10
NetworkCategory      : Public
DomainAuthenticationKind : None
IPv4Connectivity     : NoTraffic
IPv6Connectivity     : NoTraffic

PS C:\Windows\system32> Set-NetConnectionProfile -NetworkCategory Private
PS C:\Windows\system32>
```



```
Administrator: Command Prompt
C:\Windows\system32> winrm quickconfig
WinRM service is already running on this machine.
WinRM is not set up to allow remote access to this machine for management.
The following changes must be made:

Configure LocalAccountTokenFilterPolicy to grant administrative rights remotely to local users.
Make these changes [y/n]? y
WinRM has been updated for remote management.
Configured LocalAccountTokenFilterPolicy to grant administrative rights remotely to local users.
C:\Windows\system32>
```


Step 13 - Testing Remote Access using Evil-WinRM

In this step, I attempted to connect from the Kali Linux machine to the Windows target using **Evil-WinRM**. The purpose of this step was to test whether the WinRM service configured in the previous steps could be accessed remotely.

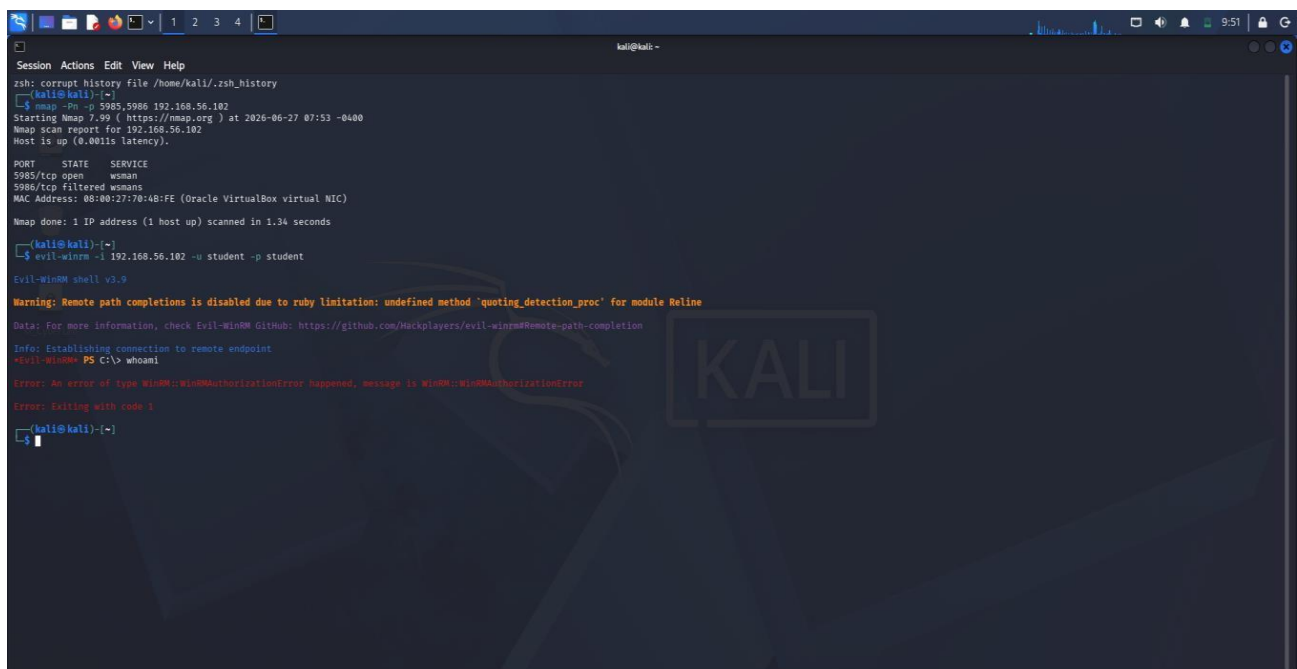
I used the following command: **evil-winrm -i 192.168.56.102 -u student -p student**
I used **Evil-WinRM** because it allows a user to connect to a Windows machine remotely through the WinRM service.

In this command:

- **-i 192.168.56.102** specifies the IP address of the Windows target machine.
- **-u student** specifies the username used for the connection.
- **-p student** specifies the password for that user account.

After running the command, Evil-WinRM attempted to establish a connection with the Windows machine. However, the connection was not successful. The system displayed a **WinRM Authorization Error**.

The error showed that although the WinRM service was configured and the connection attempt reached the target, the provided user account did not have the required permission to access the Windows machine through WinRM. The command then exited with an error.



```
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$
$ nmap -Pn -p 5985,5986 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-27 07:53 -0400
Nmap scan report for 192.168.56.102
Host is up (0.0011s latency).

PORT      STATE SERVICE
5985/tcp  open  wman
5986/tcp  filtered wmans
MAC Address: 08:00:27:70:48:FE (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.34 seconds
(kali@kali)~$
$ evil-winrm -i 192.168.56.102 -u student -p student
Evil-WinRM shell v3.9

Warning: Remote path completions is disabled due to ruby limitation: undefined method 'quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM Github: https://github.com/Hackplayers/evil-winrm#remote-path-completion
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\> whoami
Error: An error of type WinRM:WinRMAuthorizationError happened, message is WinRM:WinRMAuthorizationError
Error: Exiting with code 1
(kali@kali)~$
```

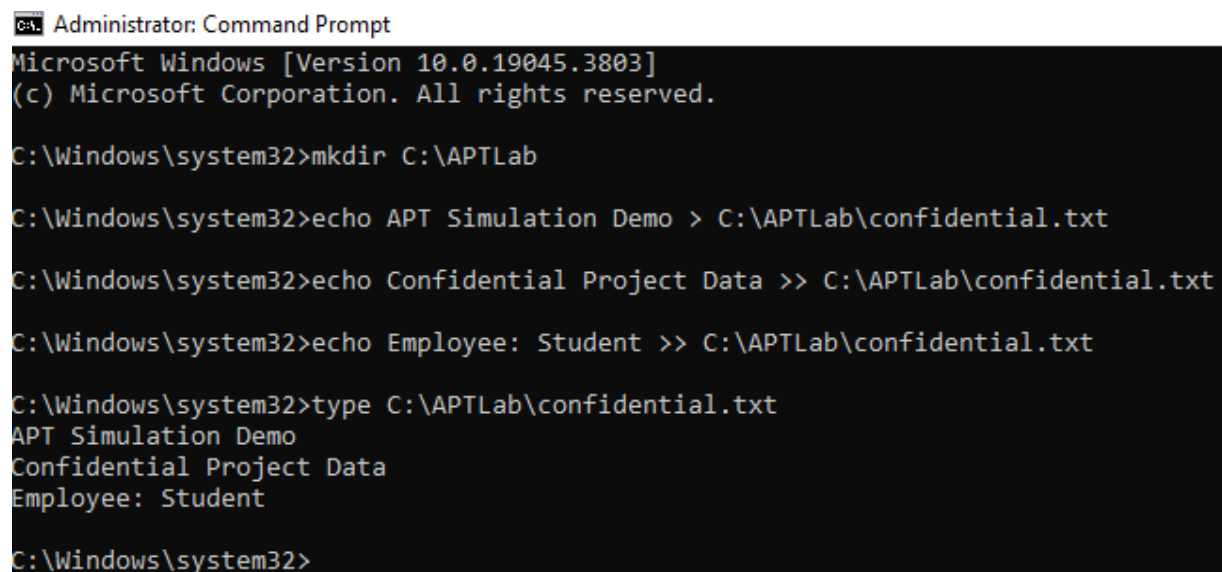
Step 14 - Creating Sample Confidential Data

After completing the WinRM connection test, I created a sample folder named **APTLab** on the Windows machine. This folder was used to store some example confidential data for the simulation.

First, I used the following command: **mkdir C:\APTLab**
This command created a new folder named **APTLab** in the **C** drive.

Next, I created a file named **confidential.txt** inside this folder and added some sample information to it. The file contained example data such as **APT Simulation Demo**, **Confidential Project Data** and **Employee: Student**.

Finally, I used the type command to display the contents of the file and confirm that the sample data was created successfully.



```
C:\> Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>mkdir C:\APTLab

C:\Windows\system32>echo APT Simulation Demo > C:\APTLab\confidential.txt

C:\Windows\system32>echo Confidential Project Data >> C:\APTLab\confidential.txt

C:\Windows\system32>echo Employee: Student >> C:\APTLab\confidential.txt

C:\Windows\system32>type C:\APTLab\confidential.txt
APT Simulation Demo
Confidential Project Data
Employee: Student

C:\Windows\system32>
```

Step 15 - Creating a Scheduled Task for Persistence

In this step, I created a scheduled task on the Windows machine to simulate **persistence**. Persistence means maintaining a way for a program or process to run again after the attacker has gained access to the system.

I used the command:

```
schtasks /create /tn "APTPersistence" /tr "C:\Windows\System32\notepad.exe" /sc onlogon /ru student
```

This command created a scheduled task named **APTPersistence**.

After that, I used:

```
schtasks /query /tn "APTPersistence"
```

This command was used to check whether the scheduled task was created correctly. The output showed the task name **APTPersistence** and its status as **Ready**, confirming that the task was successfully added to the system.

```
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>schtasks /create /tn "APTPersistence" /tr "C:\Windows\System32\notepad.exe" /sc onlogon /ru student
SUCCESS: The scheduled task "APTPersistence" has successfully been created.

C:\Windows\system32>schtasks /query /tn "APTPersistence"

Folder: \
TaskName           Next Run Time      Status
=====
APTPersistence      N/A                Ready

C:\Windows\system32>
```

Step 16 - Simulating Data Collection

In this step, I simulated the collection of confidential data from the Windows machine. The purpose was to show how, after gaining access to a system, an attacker might copy important files to another location before attempting to move or exfiltrate the data.

First, I created a new folder named **APTEXfil** using the following command:

mkdir C:\APTEXfil

This created a separate folder on the **C drive**. I used this folder to store the copied data for the simulation.

Next, I used the command:

copy C:\APTLab\confidential.txt C:\APTEXfil\stolen_data.txt

This command copied the sample confidential file from the **APTLab** folder into the **APTEXfil** folder. The copied file was given the name **stolen_data.txt**.

The message “**1 File(s) copied.**” confirmed that the file was copied successfully.

Finally, I used: **type C:\APTEXfil\stolen_data.txt**

This command displayed the contents of the copied file.

```
C:\Windows\system32>mkdir C:\APTEXfil

C:\Windows\system32>copy C:\APTLab\confidential.txt C:\APTEXfil\stolen_data.txt
1 file(s) copied.

C:\Windows\system32>type C:\APTEXfil\stolen_data.txt
APT Simulation Demo
Confidential Project Data
Employee: Student

C:\Windows\system32>
```

Step 17 - Opening Event Viewer to monitor System Activity

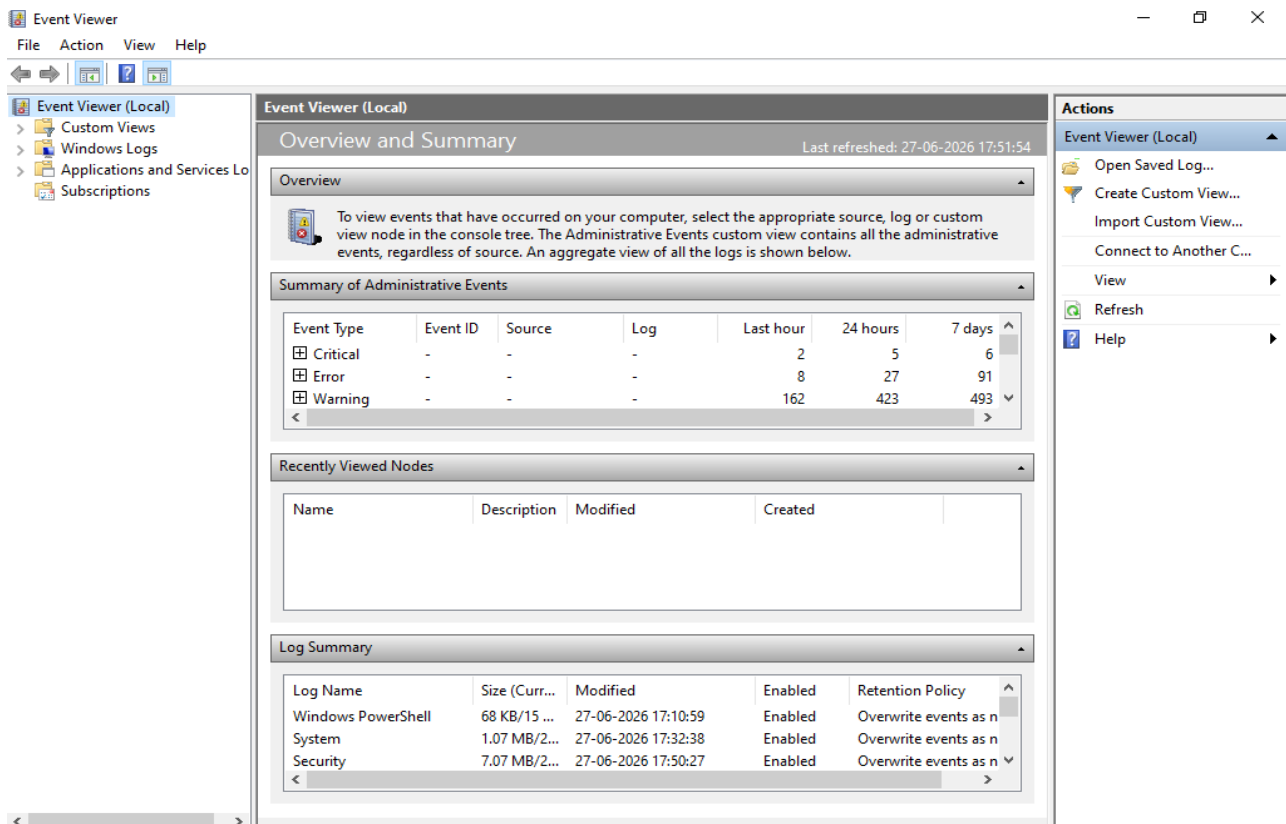
In this step, I opened **Windows Event Viewer** on the Windows machine. Event Viewer is a built-in Windows tool that records different activities and events happening in the system.

I opened Event Viewer to check the available system logs after performing the previous APT simulation steps, such as creating a scheduled task and copying the sample confidential file.

The Event Viewer window shows different sections including:

- **Windows Logs** - Contains important system, security, application and other Windows events.
- **Applications and Services Logs** - Contains logs related to specific Windows services and applications.
- **Security Logs** - Can contain security-related events such as user activity and other monitored actions.
- **System Logs** - Contains events related to Windows services, drivers and system operations.

The log summary also showed that Windows logging was enabled and events were being recorded.



Step 18 - Running the Scheduled Task and Checking its Activity

In this step, I manually ran the scheduled task that I created earlier, named **APTPersistence**. The purpose of this step was to check whether the scheduled task could run successfully and to observe its activity through Windows Event Viewer.

I used the following command: `schtasks /run /tn "APTPersistence"`

This command was used to start the scheduled task manually.

- **schtasks** - Windows command used to manage scheduled tasks.
- **/run** - Tells Windows to run the selected task immediately.
- **/tn "APTPersistence"** - Specifies the name of the scheduled task that should be executed.

After running the command, Windows displayed the message:

"SUCCESS: Attempted to run the scheduled task 'APTPersistence'."

This confirmed that Windows accepted the command and attempted to start the scheduled task.

```
C:\Windows\system32>schtasks /run /tn "APTPersistence"  
SUCCESS: Attempted to run the scheduled task "APTPersistence".  
C:\Windows\system32>
```

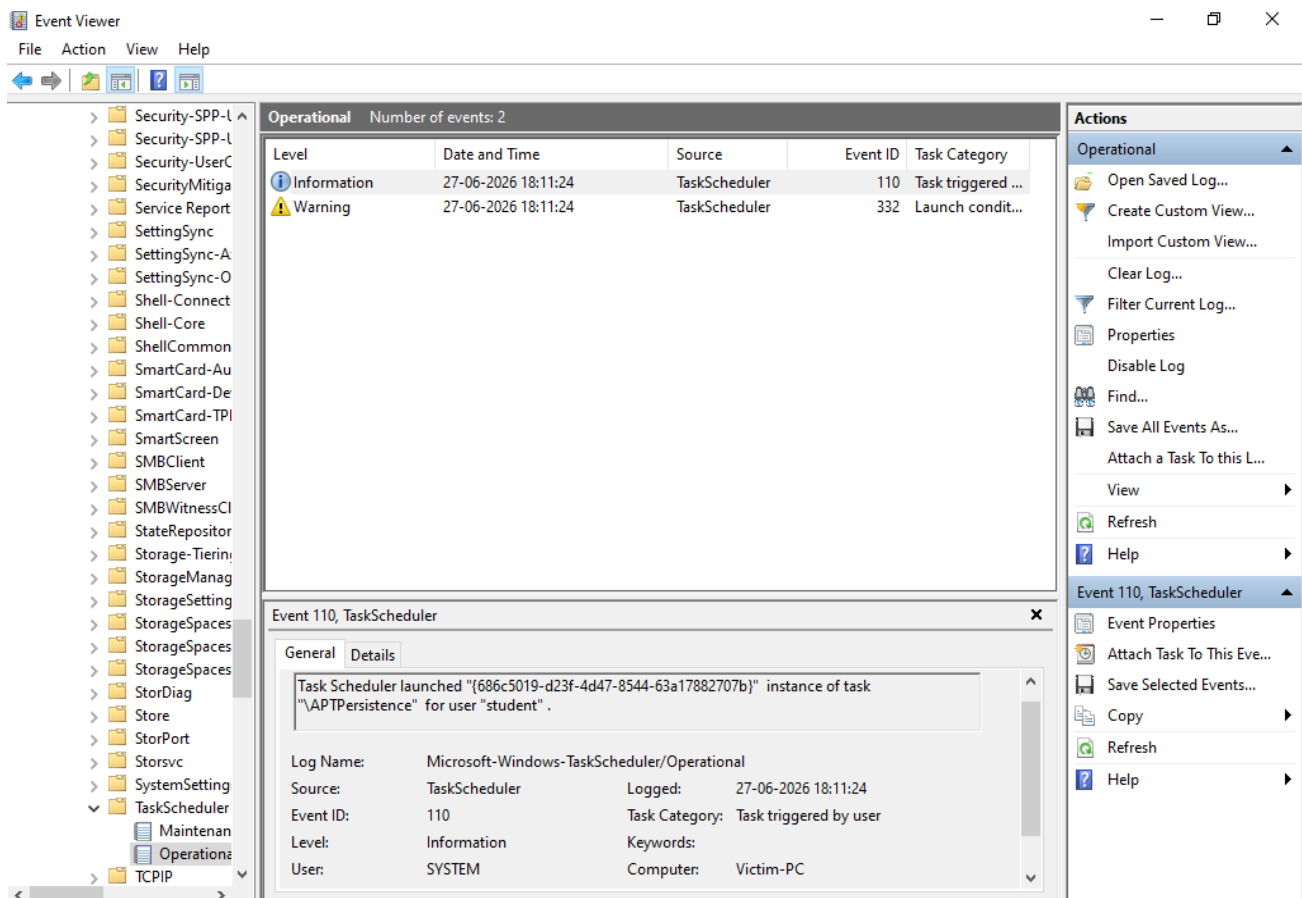

Step 19 - Verifying the Scheduled Task Execution in Event Viewer

After creating and manually running the **APTPersistence** scheduled task in the previous step, I opened **Windows Event Viewer** to check whether Windows recorded the task execution.

I navigated to the **Task Scheduler Operational log**, which stores events related to scheduled tasks. In the screenshot, the log shows **two events** generated by the Task Scheduler.

The important event shown is **Event ID 110** with the source **TaskScheduler**. The event message states that the Task Scheduler launched an instance of the task **\APTPersistence** for the user **student**. This confirms that the scheduled task was successfully triggered and Windows recorded its activity.

It also shows the date and time of the event, allowing the activity to be connected with the time when the scheduled task was executed. This is useful from a security monitoring point of view because scheduled tasks can sometimes be used to maintain persistence on a system.



Step 20 - Accessing the Elastic Security Project

In this step, I opened my **Elastic Cloud account** to access the Elastic Security environment. This platform was used for the Blue Team monitoring and investigation part of the APT attack simulation.

The Elastic Cloud home page showed my existing serverless project named **“My Security project.”** The project status was shown as complete, which indicated that the Elastic Security environment was already set up and ready to use.

I selected the **Open** option to enter the security project and continue with the monitoring process.

The screenshot displays the Elastic Cloud dashboard. At the top, the 'elastic' logo is on the left, and a 'Trial - 9 days left' badge is on the right. Below the header, the breadcrumb 'Org-14b1a6e / Cloud home' is visible. A left sidebar contains navigation links: Home, Hosted, Serverless, Connected, Security, Extensions, Organization, and Billing. The main content area is titled 'Welcome to Elastic Cloud' and features three primary sections: 'Hosted deployments' (with a 'Create hosted deployment' button), 'Serverless projects' (containing a table with one project, 'My Security project', in a 'Complete' state, with 'Open' and 'Manage' actions), and 'Connected clusters' (with a 'Connect self-managed cluster' button). A right sidebar includes 'News' and 'Community' sections with various updates and links.

Project	Type	Cloud provider & region	Actions
My Security project	Security Complete	GCP - US Central 1 (Iowa)	Open Manage

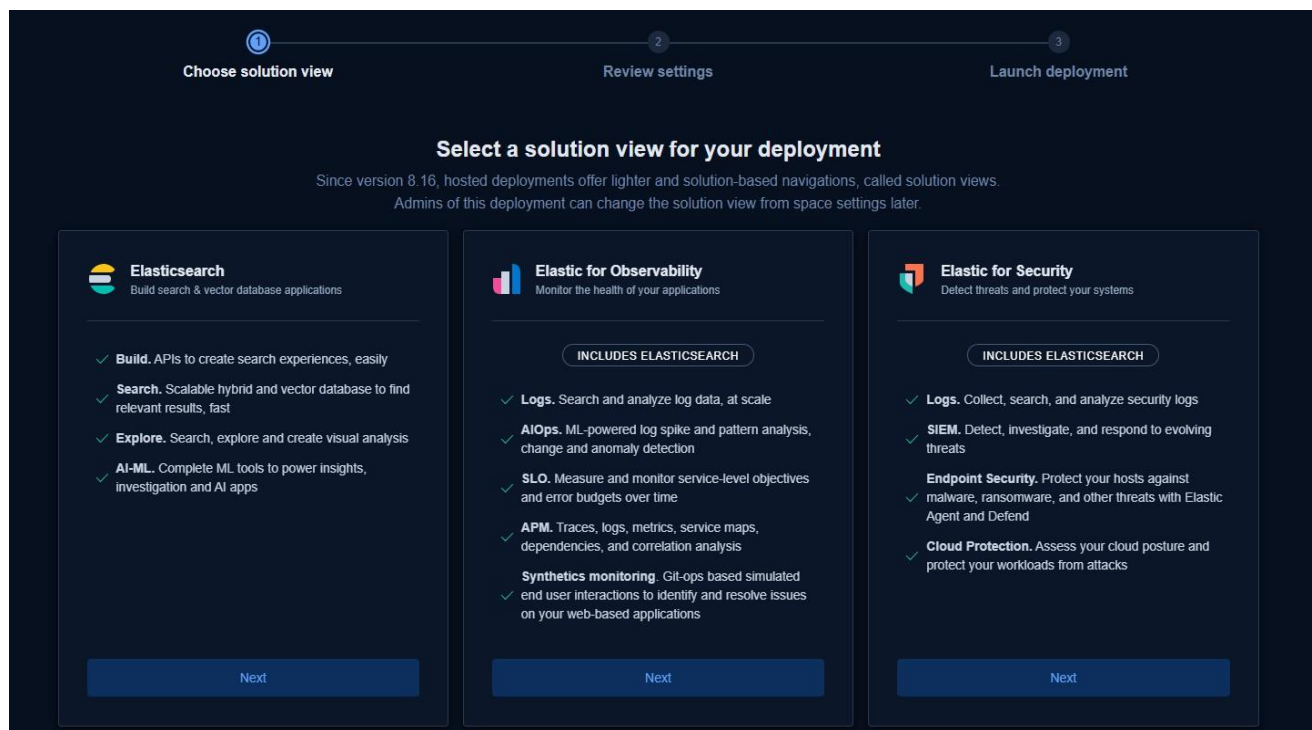
Step 21 - Selecting the Elastic Security Solution

After accessing the Elastic Security project in the previous step, the next step was to select the appropriate **solution** view for the deployment.

The Elastic Cloud page displayed three main options:

- **Elasticsearch** - mainly used for search, data storage and analysis.
- **Elastic for Observability** - used for monitoring applications, logs, performance and system health.
- **Elastic for Security** - used for security monitoring, threat detection, investigation and incident response.

Since this project focuses on cybersecurity and security event monitoring, I selected **Elastic for Security**. After selecting it, I clicked **Next** to move to the **Review Settings** step and continue the deployment.



Step 22 - Reviewing and Creating the Elastic Cloud Deployment

In this step, I reviewed the settings before creating the Elastic Security deployment. This page allowed me to check that the correct options were selected for my security environment.

First, I entered a name for the deployment. The following settings were shown:

- **Cloud Provider:** Google Cloud
- **Region:** Iowa, US (us-central1)
- **Hardware Profile:** Storage optimized
- **Version:** Elastic version 9.4.2 (latest)

I reviewed these settings to make sure they were suitable for the project. The cloud provider and region decide where the Elastic deployment and its data will be hosted. After checking all the settings, I clicked **Create hosted deployment**.

The screenshot shows the 'Review settings' step in the Elastic Cloud deployment wizard. At the top, there are three progress indicators: 'Choose solution view' (completed), 'Review settings' (current step), and 'Launch deployment'. The main heading is 'Confirm the settings of your deployment' with a sub-note: 'Your data will be stored in the cloud provider of your choice.' Below this is a 'Security' section with a form. The 'Name' field contains 'My Security deployment'. Under 'Settings', there are four dropdown menus: 'Cloud provider' (Google Cloud), 'Region' (us Iowa (us-central1)), 'Hardware profile' (Storage optimized), and 'Version' (9.4.2 (latest)). There is a link for '> Advanced settings' and a status 'ML nodes autoscaling enabled'. At the bottom, there is a 'Show configuration' link and a 'Create hosted deployment' button.

Choose solution view Review settings Launch deployment

Confirm the settings of your deployment

Your data will be stored in the cloud provider of your choice.

 **Security**

Name

My Security deployment

Settings

Cloud provider | Google Cloud

Region | us Iowa (us-central1)

Hardware profile ⓘ | Storage optimized

Version ⓘ | 9.4.2 (latest)

> Advanced settings ML nodes autoscaling enabled ⓘ

Show configuration ^ Create hosted deployment

Step 23 - Saving the Elastic Deployment Credentials

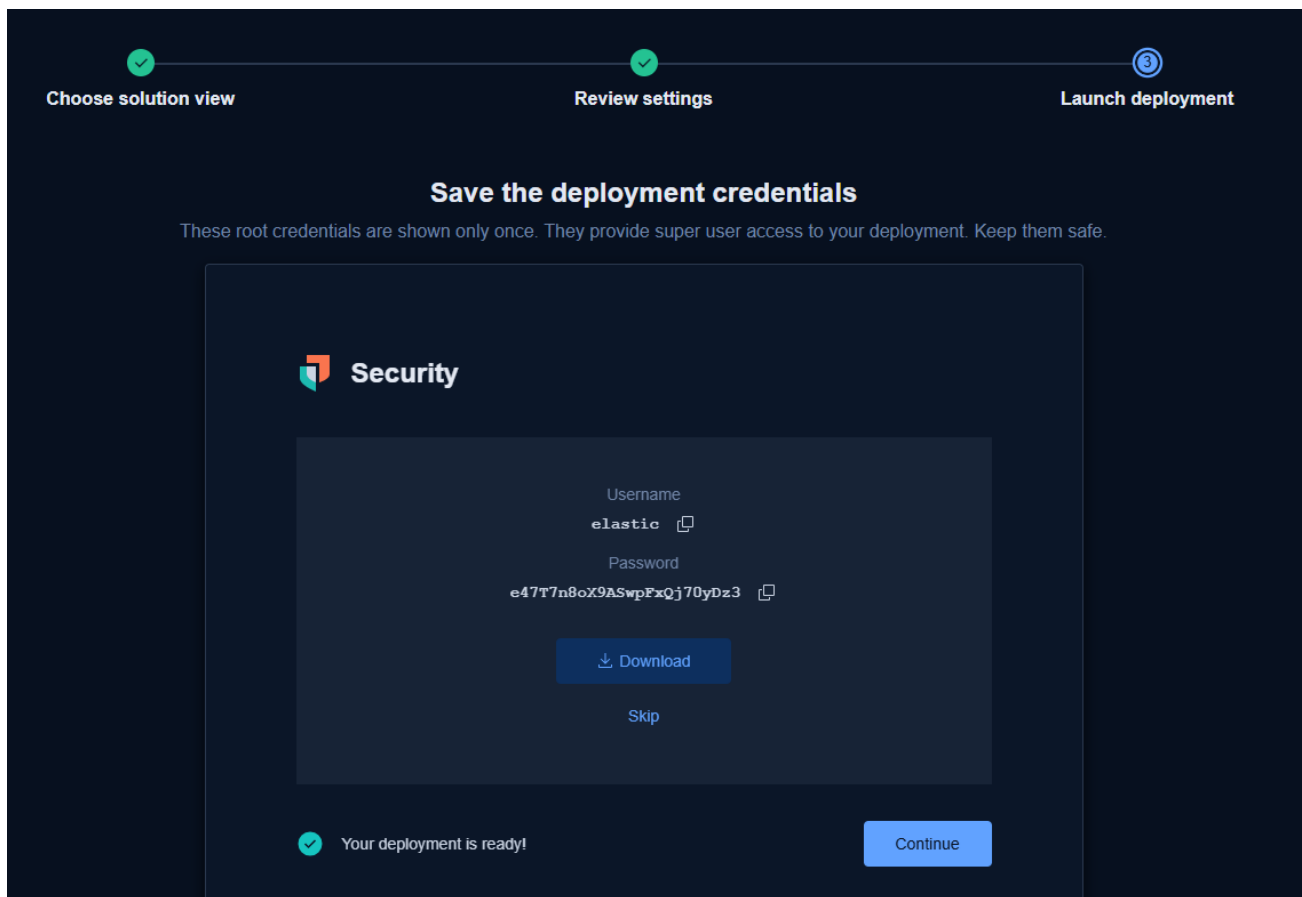
In this step, the Elastic Security deployment was successfully created and was ready to use. Elastic then displayed the **deployment credentials**, including the username and password.

The username shown was **elastic**. This account can be used to access and manage the Elastic Security environment.

The password was also displayed on this page. These credentials are important because they are required to log in to the deployment.

I saved the credentials using the **Download** option. This helps keep the login details available for later use. After saving the credentials, I clicked **Continue** to move to the newly created Elastic Security deployment.

The message “**Your deployment is ready!**” confirms that the Elastic Security environment was created successfully and is ready for use.



Step 24 - Accessing the Elastic Security Dashboard

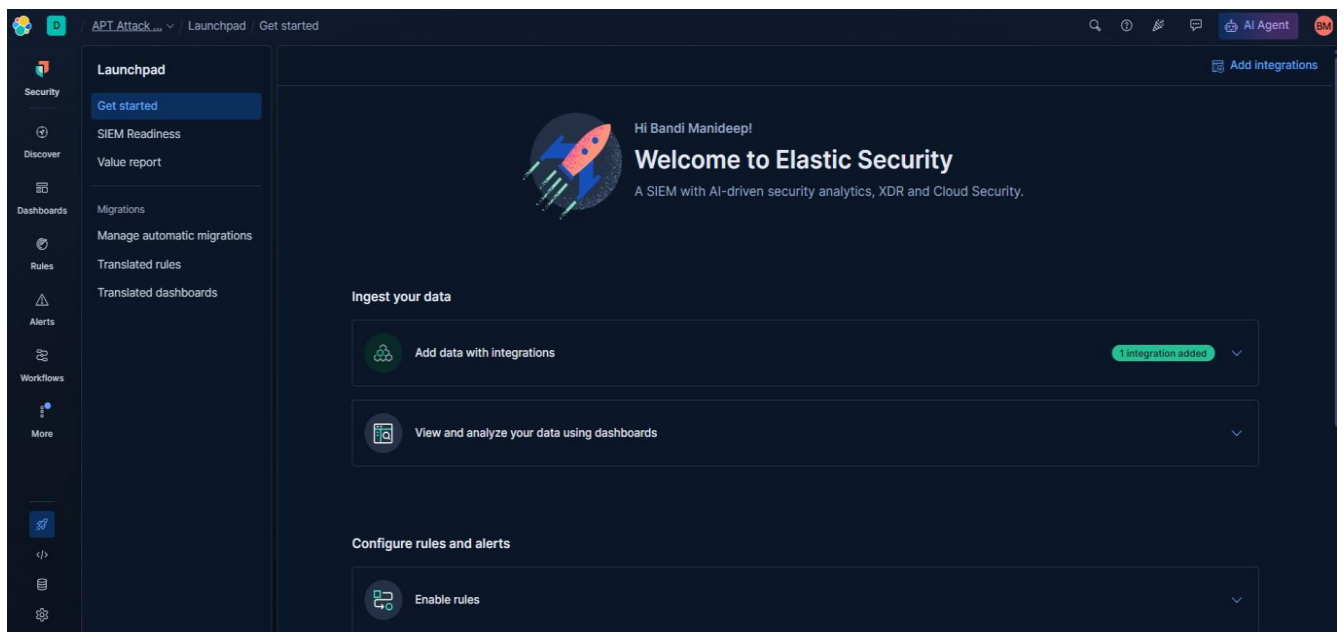
After saving the deployment credentials and completing the Elastic Security setup, I accessed the **Elastic Security dashboard**. This is the main page where the security environment can be managed.

The dashboard shows different sections that can be used during the project.

- **Add data with integrations:** This option is used to connect data sources and send logs or security data to Elastic. In this project, this is important for collecting events that can later be monitored and analyzed.
- **View and analyze your data using dashboards:** This section helps in viewing the collected data in an organized way.
- **Enable rules:** This option is used to activate security detection rules. These rules can identify suspicious activities and generate alerts.

The left-side menu also provides options such as **Discover**, **Dashboards**, **Rules**, **Alerts** and **Workflows**. These features can be used to search logs, create visualizations, configure detection rules and investigate security alerts.

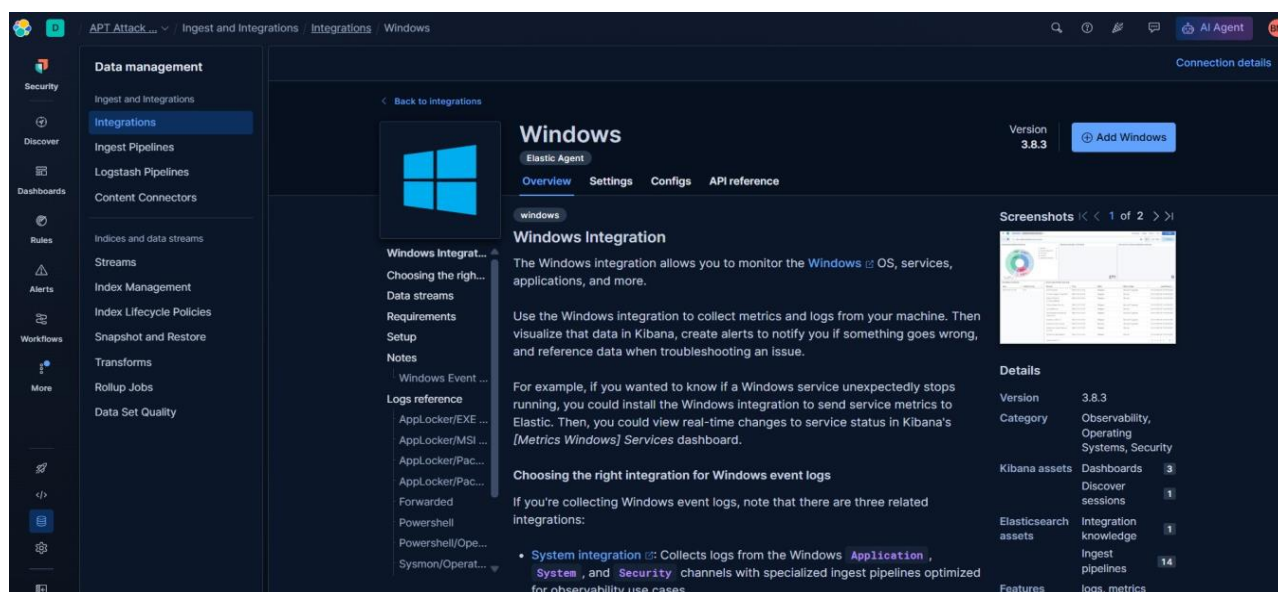
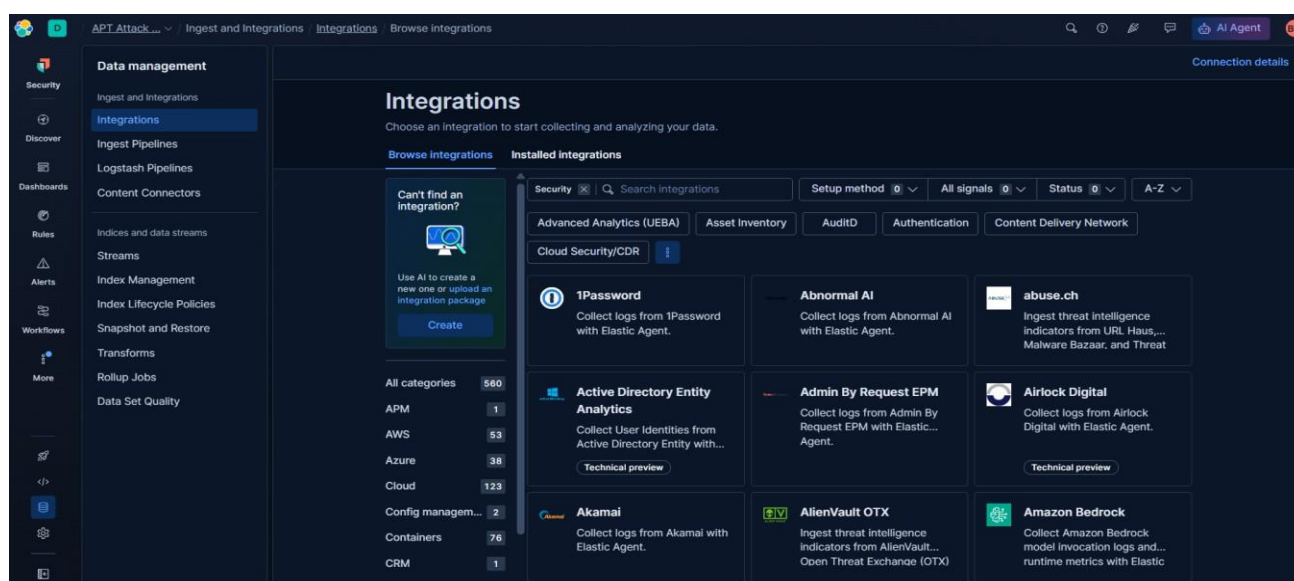
This page confirms that the **Elastic Security environment was successfully accessed and is ready for further configuration.**



Step 25 - Browsing Available Integrations

After entering the **Integrations** section, I opened the list of available integrations provided by Elastic. This page contains different integration options that can be used to collect logs, security events, threat intelligence, cloud data, and other useful information from different systems and services.

In this step, I searched for “**Windows**” in the Elastic Security **Integrations** page. The purpose of searching for Windows was to find the appropriate integration that can collect logs and security-related events from my Windows victim machine. These logs can later be sent to Elastic Security for monitoring and analysis.



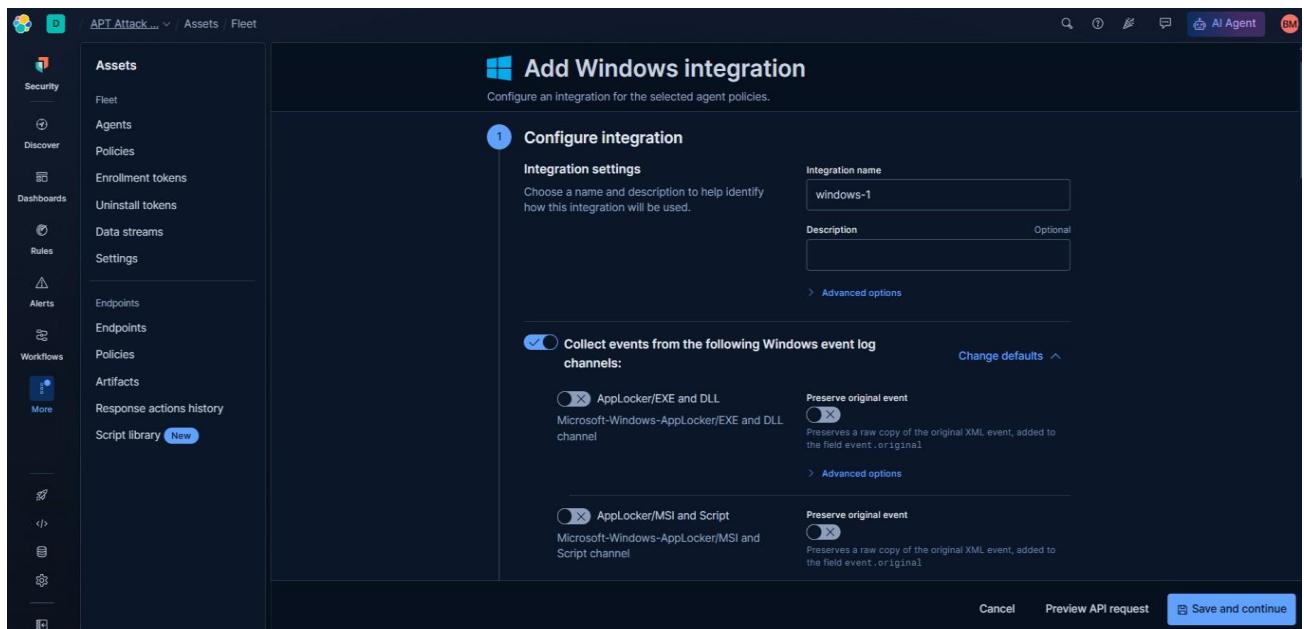
Step 26 - Configuring the Windows Integration

In this step, I configured the **Windows integration** in Elastic so that Windows event logs can be collected from the target system.

First, I gave the integration a name, **windows-1**, to easily identify it later. The description field was left empty because it was optional.

Next, I enabled the option **“Collect events from the following Windows event log channels.”** This allows Elastic to collect Windows event logs that can be useful for monitoring and security analysis.

After reviewing the available settings, I clicked **“Save and continue”** to save the Windows integration configuration.



Step 27 - Installing Elastic Agent on the Windows System

In this step, I moved to the **Add Agent** section in Elastic to install the **Elastic Agent** on my Windows machine. The Elastic Agent is used to collect system and security logs from the Windows system and send them to my Elastic Security project.

The page showed the integration policies that were already added, including the **System** and **Windows** integrations. This means the agent will use these integrations after it is installed and enrolled.

Since my target machine is running Windows, I selected the **Windows installation option**. Elastic then provided a set of PowerShell commands for downloading, extracting, installing, enrolling and starting the Elastic Agent.

The installation commands shown on this page include the enrollment information required to connect the Windows machine with my Elastic deployment.

The screenshot displays the Elastic Security console interface. On the left, a sidebar menu includes sections like Security, Discover, Dashboards, Rules, Alerts, Workflows, Agents, and Attack discovery. The main content area is divided into two panels. The left panel, titled 'Agent policy 1', shows a list of integration policies with columns for 'Integration policy' and 'Integration'. It lists 'system-1' (System v1.20.0) and 'windows-1' (Windows v1.0.0). The right panel, titled 'Add agent', provides instructions on how to install the Elastic Agent. It includes a warning about root privileges and a section for PowerShell commands to download, extract, and install the agent. The commands are as follows:

```
$ProgressPreference = 'SilentlyContinue'
Invoke-WebRequest -Uri https://artifacts.elastic.co/downloads/beats/elastic-agent-9.4.2-windows-x86_64.zip -DestinationPath .\elastic-agent-9.4.2-windows-x86_64.zip
cd elastic-agent-9.4.2-windows-x86_64
.\elastic-agent.exe install --url=https://21871b47d889498c872e7ac2f5872
```

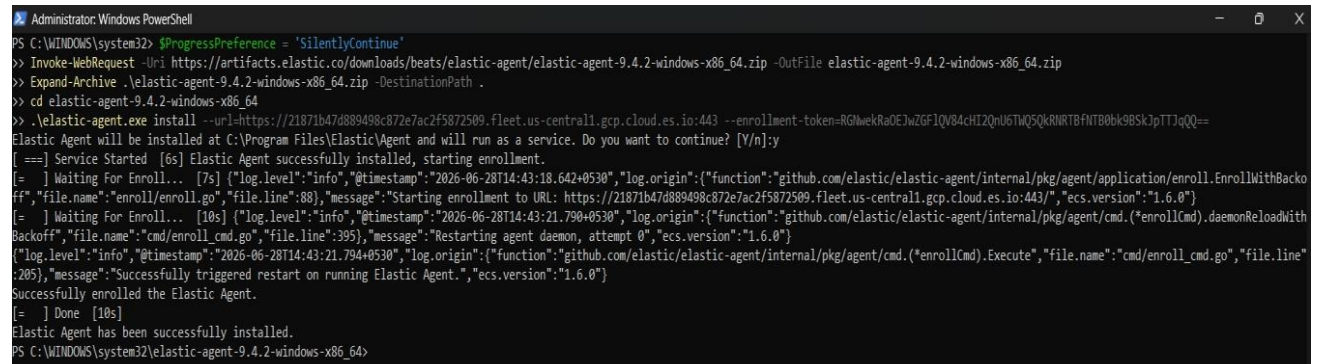
Step 28 - Installing and Enrolling the Elastic Agent

In this step, I opened **Windows PowerShell as Administrator** and ran the installation commands provided by Elastic. These commands downloaded the Elastic Agent package, extracted the files, and started the installation process.

During the installation, I used the **elastic-agent install** command with the Elastic Fleet URL and enrollment token. This enrollment process connects the Windows machine to my Elastic Security project.

The installation asked for confirmation because the Elastic Agent would be installed in **C:\Program Files\Elastic\Agent** and run as a Windows service. I entered **Y** to continue with the installation.

The output then showed that the enrollment process was started successfully. After a few seconds, the message **“Successfully enrolled the Elastic Agent”** appeared. Finally, the system displayed **“Elastic Agent has been successfully installed.”**



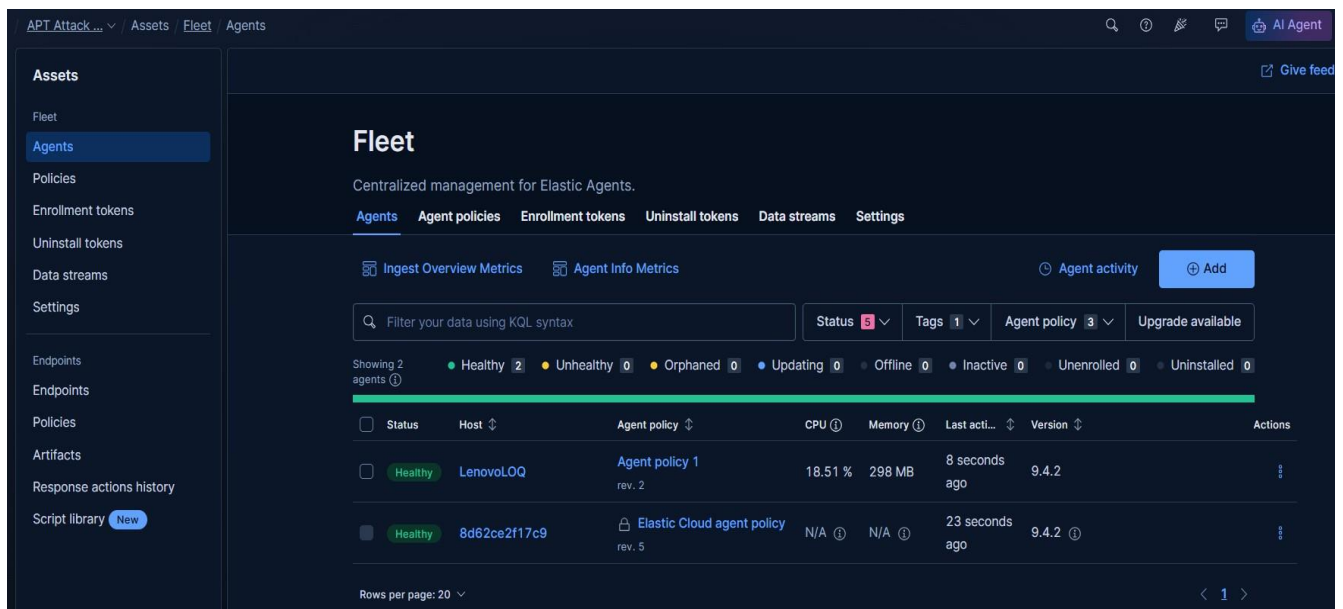
```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> $ProgressPreference = 'SilentlyContinue'
>> Invoke-WebRequest -Uri https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.4.2-windows-x86_64.zip -Outfile elastic-agent-9.4.2-windows-x86_64.zip
>> Expand-Archive .\elastic-agent-9.4.2-windows-x86_64.zip -DestinationPath .
>> cd elastic-agent-9.4.2-windows-x86_64
>> .\elastic-agent.exe install --url=https://21871b47d889498c872e7ac2f5872509.fleet.us-central1.gcp.cloud.es.io:443 --enrollment-token=RGWwEkrRa0EJwZ6FIQV84cHIZQnU6TWQ5QkNRNTBfNTB0bk9BSk3pTTJqQQ==
Elastic Agent will be installed at C:\Program Files\Elastic\Agent and will run as a service. Do you want to continue? [Y/n]:y
[ == ] Service Started [6s] Elastic Agent successfully installed, starting enrollment.
[= ] Waiting For Enroll... [7s] {"log.level":"info","@timestamp":"2026-06-28T14:43:18.642+0530","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/application/enroll.EnrollWithBackoff","file.name":"enroll/enroll.go","file.line":88},"message":"Starting enrollment to URL: https://21871b47d889498c872e7ac2f5872509.fleet.us-central1.gcp.cloud.es.io:443/", "ecs.version":"1.6.0"}
[= ] Waiting For Enroll... [10s] {"log.level":"info","@timestamp":"2026-06-28T14:43:21.790+0530","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCmd).daemonReloadWithBackoff","file.name":"cmd/enroll_cmd.go","file.line":395},"message":"Restarting agent daemon, attempt 0", "ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-06-28T14:43:21.794+0530","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCmd).Execute","file.name":"cmd/enroll_cmd.go","file.line":205},"message":"Successfully triggered restart on running Elastic Agent.", "ecs.version":"1.6.0"}
Successfully enrolled the Elastic Agent.
[= ] Done [10s]
Elastic Agent has been successfully installed.
PS C:\WINDOWS\system32> .\elastic-agent-9.4.2-windows-x86_64>
```

Step 29 - Verifying the Connected Elastic Agent

In this step, I returned to the **Fleet** → **Agents** section in Elastic to check whether the Elastic Agent installed on my Windows system was successfully connected.

The Agents page showed **two agents**, and both were marked as **Healthy**. One of the agents was running on my Windows system, **LenovoLOQ**, and it was assigned to **Agent policy 1**. This policy contains the integrations that I configured earlier, including the Windows integration.

The page also showed that the agent was recently active and running Elastic Agent version **9.4.2**. The **Healthy** status confirms that the agent was properly installed, enrolled, and successfully communicating with Elastic.



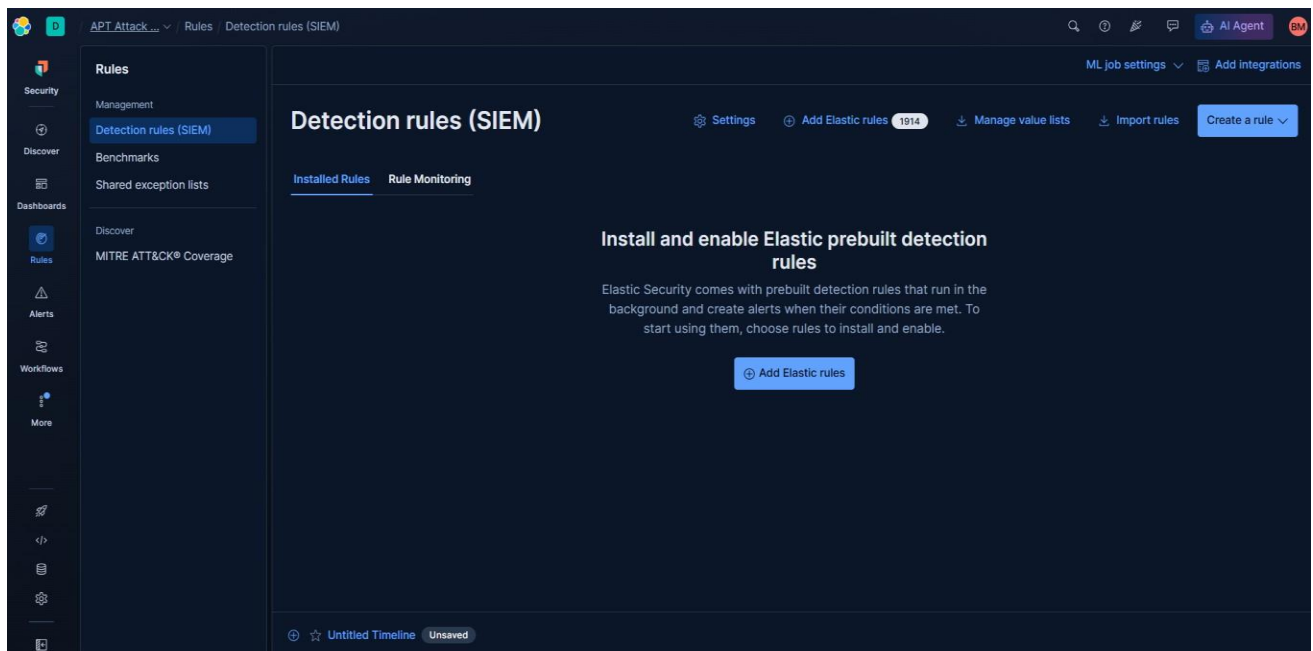
The screenshot displays the Elastic Fleet management interface. The left sidebar contains navigation links for Assets, Fleet, Agents, Policies, Enrollment tokens, Uninstall tokens, Data streams, Settings, Endpoints, Artifacts, Response actions history, and Script library. The main content area is titled 'Fleet' and 'Centralized management for Elastic Agents.' It includes tabs for Agents, Agent policies, Enrollment tokens, Uninstall tokens, Data streams, and Settings. Below these tabs are metrics for Ingest Overview and Agent Info, along with an 'Agent activity' button. A search bar allows filtering by KQL syntax. A status filter shows 2 Healthy agents. A table lists the agents with columns for Status, Host, Agent policy, CPU, Memory, Last activity, Version, and Actions. The first agent is 'LenovoLOQ' with status 'Healthy', assigned to 'Agent policy 1', running CPU 18.51% and Memory 298 MB, last active 8 seconds ago, version 9.4.2. The second agent is '8d62ce2f17c9' with status 'Healthy', assigned to 'Elastic Cloud agent policy', showing N/A for CPU and Memory, last active 23 seconds ago, version 9.4.2. The bottom of the page shows 'Rows per page: 20' and a pagination control.

Status	Host	Agent policy	CPU	Memory	Last acti...	Version	Actions
Healthy	LenovoLOQ	Agent policy 1 rev. 2	18.51 %	298 MB	8 seconds ago	9.4.2	
Healthy	8d62ce2f17c9	Elastic Cloud agent policy rev. 5	N/A	N/A	23 seconds ago	9.4.2	

Step 30 - Opening Detection Rules in Elastic Security

In this step, I opened the **Detection Rules (SIEM)** section in Elastic Security. This page is used to manage security detection rules that monitor the collected data and generate alerts when suspicious activity is detected.

The screen showed an option to **Add Elastic rules**. These are prebuilt detection rules provided by Elastic. I reached this page to install and enable the required rules so that the system can automatically monitor security events and create alerts when the rule conditions are matched.



Step 31 - Viewing and Selecting Elastic Detection Rules

In this step, I opened the **Add Elastic rules** page in Elastic Security. This page shows many prebuilt detection rules provided by Elastic. These rules are designed to identify suspicious activities and possible security threats.

I reviewed the available rules and checked their details, such as the **rule name**, **risk score**, and **severity level**. The list included rules related to Windows activities, PowerShell, unusual user behavior, services, privilege escalation, and other suspicious actions.

After reviewing the rules, I clicked the **Install all** button to install all the available prebuilt detection rules at once. By doing this, the rules were added to my Elastic Security environment without installing them one by one.

The screenshot shows the 'Add Elastic rules' interface in Elastic Security. The left sidebar contains navigation options: Security, Management, Detection rules (SIEM), Benchmarks, Shared exception lists, Discover, MITRE ATT&CK® Coverage, Rules, Alerts, Workflows, and More. The main content area is titled 'Add Elastic rules' and features a search bar, a table of rules, and an 'Install all' button. The table lists various rules with their respective risk scores and severity levels.

Rule	Risk score	Severity
Startup/Logon Script added to Group Policy Object	47	Medium
Persistence via a Windows Installer	47	Medium
Suspected Lateral Movement from Compromised Host	73	High
AWS Bedrock Detected Multiple Attempts to use Denied Models by a Single	73	High
Unusual Windows Username	21	Low
Unusual Windows Service	21	Low
Suspicious Powershell Script	21	Low
Unusual Windows User Privilege Elevation Activity	21	Low
Unusual Windows Remote User	21	Low
Spike in User Lifecycle Management Change Events	21	Low
Systemd Service Created	47	Medium

Step 32 - Verifying the Installed Detection Rules

In this step, I returned to the **Detection rules (SIEM)** page after installing all the Elastic prebuilt rules.

The page now shows **1914 installed rules**, confirming that the rules were successfully added to the Elastic Security environment. I could see the different detection rules along with details such as their **risk score**, **severity level**, **last run time**, **last response**, and **status**.

Some rules were also shown as **enabled**, meaning they are active and ready to monitor the security data collected from the system. These rules can automatically check for suspicious activities and generate alerts when their conditions are matched.

The screenshot displays the Elastic Security interface for the 'Detection rules (SIEM)' page. The left sidebar shows navigation options: Security, Management, Detection rules (SIEM), Benchmarks, Shared exception lists, Discover, Rules, MITRE ATT&CK® Coverage, Alerts, Workflows, and More. The main content area is titled 'Detection rules (SIEM)' and includes tabs for 'Installed Rules' (1914) and 'Rule Monitoring' (1914). A search bar is present with filters for 'Tags' (237), 'Last response' (3), and 'Elastic rules (1914)'. A table lists the first 20 of 1914 rules, showing columns for Rule, Risk score, Severity, Last run, Last response, Last updated, Notify, and Enable. The rules listed include 'Container Workload Protection', 'Endpoint Security (Elastic Defend)', 'Potential Widespread Malware Infection Across Multiple Ho...', 'Newly Observed Palo Alto Network Alert', 'Account Discovery Command via SYSTEM Account', 'Exploit - Prevented - Elastic Endgame', and 'AWS STS Role Assumption by User'.

Rule	Risk score	Severity	Last run	Last response	Last updated	Notify	Enable
Container Workload Protection	47	Medium	25 seconds ago	Warning	3 hours ago	On	On
Endpoint Security (Elastic Defend)	47	Medium	1 minute ago	Warning	3 hours ago	On	On
Potential Widespread Malware Infection Across Multiple Ho...	73	High	—	—	3 hours ago	On	On
Newly Observed Palo Alto Network Alert	99	Critical	—	—	3 hours ago	On	On
Account Discovery Command via SYSTEM Account	21	Low	—	—	3 hours ago	On	On
Exploit - Prevented - Elastic Endgame	47	Medium	—	—	3 hours ago	On	On
AWS STS Role Assumption by User	21	Low	—	—	3 hours ago	On	On

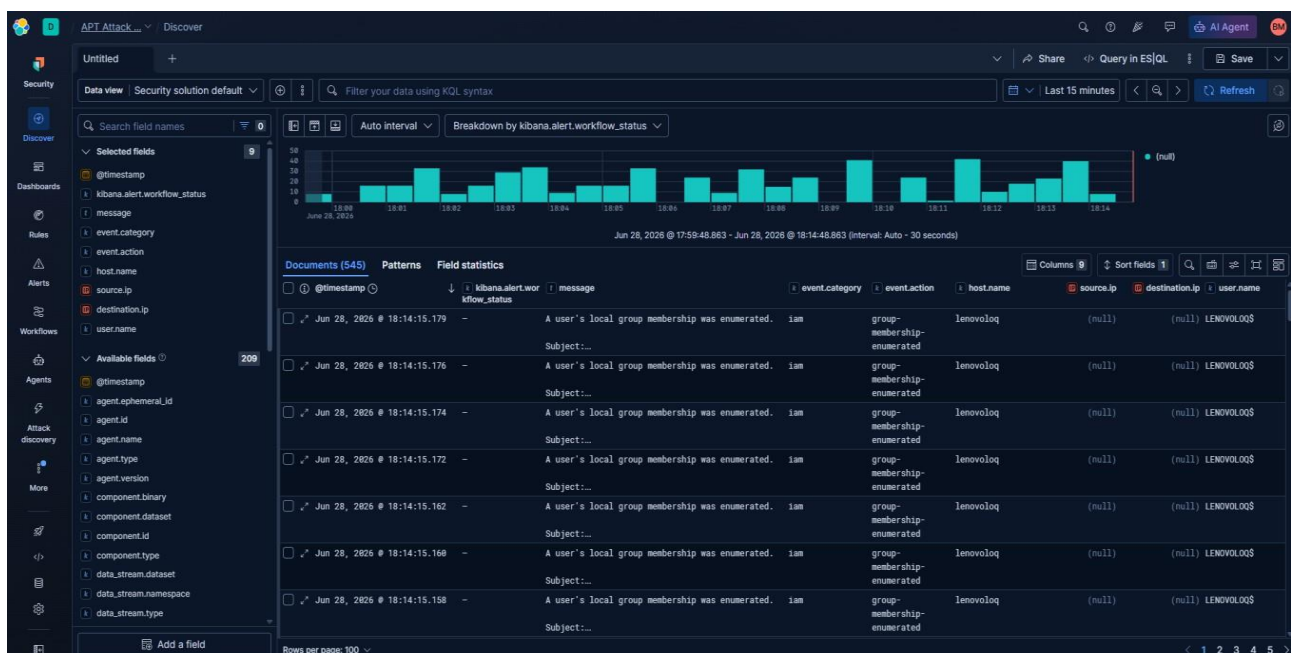
Step 33 - Analyzing Collected Security Events

In this step, I opened the **Discover** section in Elastic Security to check whether the Windows system events were being collected successfully.

I could see **545 documents (events)** in the selected time range. The events were coming from my Windows system, with the host name shown as **lenovo10q**. The collected data included information such as the **timestamp**, **message**, **event category**, **event action**, **host name**, **source IP**, **destination IP** and **user name**.

The displayed events showed activities such as a **user's local group membership being enumerated**, which means the system recorded information about checking or listing the groups associated with a user.

This step confirmed that the **Elastic Agent** and **Windows integration** were successfully **collecting and sending security event data** to Elastic Security.



Step 34 - Performing Test Activities to generate Security Events

In this step, I performed multiple activities in **Windows PowerShell** to generate different security events.

First, I used the command: **net user attacker Password@123 /add**

This command created a new Windows user account named **attacker** with the specified password. After running the command, the message “**The command completed successfully**” confirmed that the new user account was created successfully.

Next, I used the command: **net localgroup Administrators attacker /add**

This command added the **attacker** user account to the local **Administrators** group. The successful completion message confirmed that the user was added to the Administrators group.

After that, I created a scheduled task using the command:

schtasks /create /tn "APTTest" /tr "notepad.exe" /sc onlogon

This command created a scheduled task named **APTTest**. The task was configured to start **Notepad (notepad.exe)** whenever a user logs on to the system. The message “**SUCCESS: The scheduled task 'APTTest' has successfully been created**” confirmed that the task was created.

Next, I ran: **powershell -ExecutionPolicy Bypass**

This opened PowerShell with the **Execution Policy Bypass** option. This allowed PowerShell commands or scripts to run without being restricted by the normal execution policy settings.

Finally, I used the command: **Get-Process**

This command displayed the processes currently running on the Windows system.

```
PS C:\WINDOWS\system32\elastic-agent-9.4.2-windows-x86_64> net user attacker Password@123 /add
The command completed successfully.

PS C:\WINDOWS\system32\elastic-agent-9.4.2-windows-x86_64> net localgroup Administrators attacker /add
The command completed successfully.

PS C:\WINDOWS\system32\elastic-agent-9.4.2-windows-x86_64> schtasks /create /tn "APTTest" /tr "notepad.exe" /sc onlogon
SUCCESS: The scheduled task "APTTest" has successfully been created.
PS C:\WINDOWS\system32\elastic-agent-9.4.2-windows-x86_64> powershell -ExecutionPolicy Bypass
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32\elastic-agent-9.4.2-windows-x86_64> Get-Process

Handles  NPM(K)    PM(K)      WS(K)      CPU(s)     Id  SI ProcessName
-----
243      11        3312       9080        4.70      9968  0 AggregatorHost
189       8         1736       5276        0.27      4940  0 AISControlService
365      19        12668      40720       2.39     22124  3 AppActions
561      27        68264      48452       1.03     10180  3 ApplicationFrameHost
8628     16        41656     39500      1,586.53   3976  0 audiodg
233      25         6212       4892        0.47     13476  3 backgroundTaskHost
470      22        23004     58696       2.09     24304  3 backgroundTaskHost
275      17         21768     55156       0.56      2036  3 BlueStacksServices
281      16        13816     47060       2.41     5116  3 BlueStacksServices
777      35        59780     88160     113.77    15588  3 BlueStacksServices
474      26        60044     61668       1.53     16268  3 BlueStacksServices
338      24         9240     18056       0.80     18228  3 browserhost
463      20         5920     13828       8.25     23064  3 CastSrv
185      14        13292     20952       3.16      2132  3 chrome
369      25        46360     89308       5.19      4056  3 chrome
514      51       383776    395732     423.13    6112  3 chrome
283      26        80468     90256       42.30     8516  3 chrome
544      27        43964     42188       1.03     12236  3 chrome
476      46       599272    674092     346.66    15916  3 chrome
247      17       11384     23900       58.05    17272  3 chrome
452      36       36768     72248     157.98    18056  3 chrome
219      19       16468     31824       0.19     18204  3 chrome
385      34       92828    162980     63.69    18928  3 chrome
350      26       41728     98584       5.16     20588  3 chrome
244      12         2900     11776       0.16     22356  3 chrome
2340     86       190224    320372     594.98    24804  3 chrome
2356     61       508364    431420     2,052.25  25240  3 chrome
91       6         4052       5532       0.02      7384  3 cmd
91       6         4460       5536       0.02     23532  3 cmd
76       8         1128       1220       0.06      3952  0 conhost
100      9         1336       7804       0.03      8208  3 conhost
148      11         2140     11092       0.17      9392  3 conhost
```

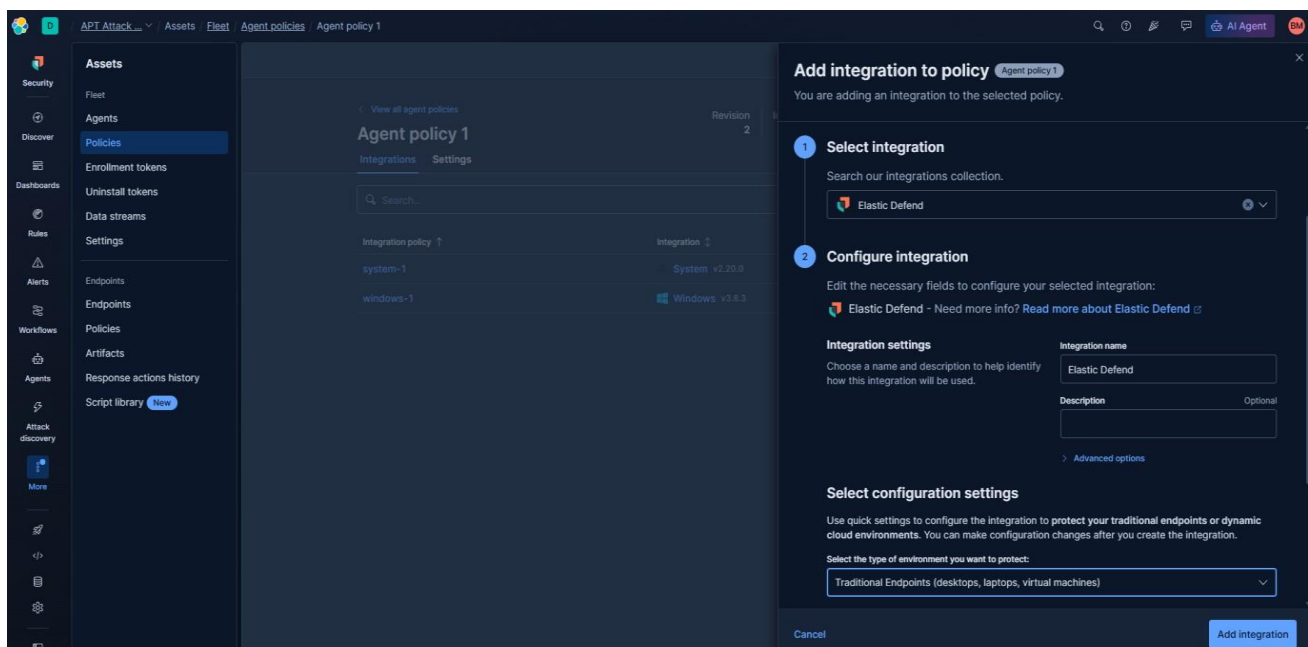
Step 35 - Adding Elastic Defend Integration for Endpoint Protection

In this step, I added the **Elastic Defend integration** to my existing **Agent policy 1**. I did this to provide additional security monitoring and protection for the Windows endpoint where the Elastic Agent was installed.

First, I opened **Fleet** → **Agent policies** and selected **Agent policy 1**. From there, I chose the option to add a new integration. In the integration search field, I selected **Elastic Defend**.

After selecting Elastic Defend, the configuration page was displayed. I kept the integration name as **Elastic Defend** and reviewed the available configuration options. I also selected **Traditional Endpoints**, which is suitable for protecting normal systems such as desktops, laptops and virtual machines.

Finally, I clicked the **Add integration** button to add Elastic Defend to the agent policy.



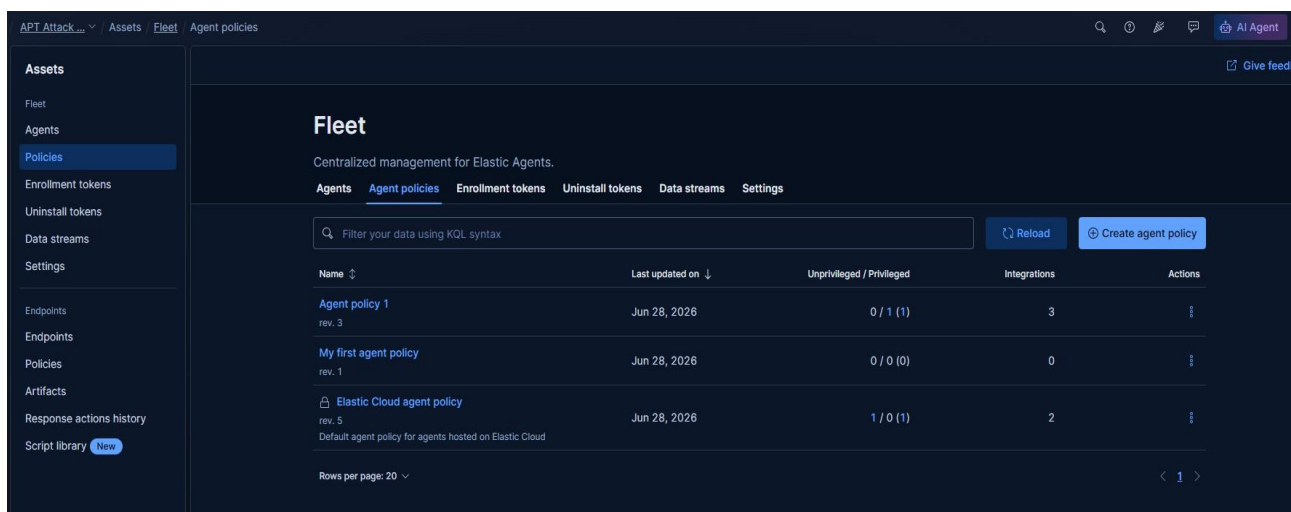
Step 36 - Viewing and Managing Agent Policies

In this step, I returned to the **Fleet** → **Agent policies** page to view the available agent policies in my Elastic Security environment. This page shows the policies that control how Elastic Agents are configured and what integrations or security features are assigned to them.

The screen displays three agent policies:

- **Agent policy 1**
- **My first agent policy**
- **Elastic Cloud agent policy**

For each policy, I can see important information such as the last updated date, the number of unprivileged and privileged agents and the number of integrations connected to that policy.



The screenshot shows the Elastic Security Fleet Agent policies page. The left sidebar contains navigation links for Assets, Fleet, Agents, Policies (selected), Enrollment tokens, Uninstall tokens, Data streams, Settings, Endpoints, Policies, Artifacts, Response actions history, and Script library. The main content area is titled 'Fleet' and 'Centralized management for Elastic Agents.' It includes tabs for Agents, Agent policies (selected), Enrollment tokens, Uninstall tokens, Data streams, and Settings. A search bar with the placeholder 'Filter your data using KQL syntax' is present, along with 'Reload' and 'Create agent policy' buttons. The table below lists the policies:

Name	Last updated on	Unprivileged / Privileged	Integrations	Actions
Agent policy 1 rev. 3	Jun 28, 2026	0 / 1 (1)	3	
My first agent policy rev. 1	Jun 28, 2026	0 / 0 (0)	0	
Elastic Cloud agent policy rev. 5 Default agent policy for agents hosted on Elastic Cloud	Jun 28, 2026	1 / 0 (1)	2	

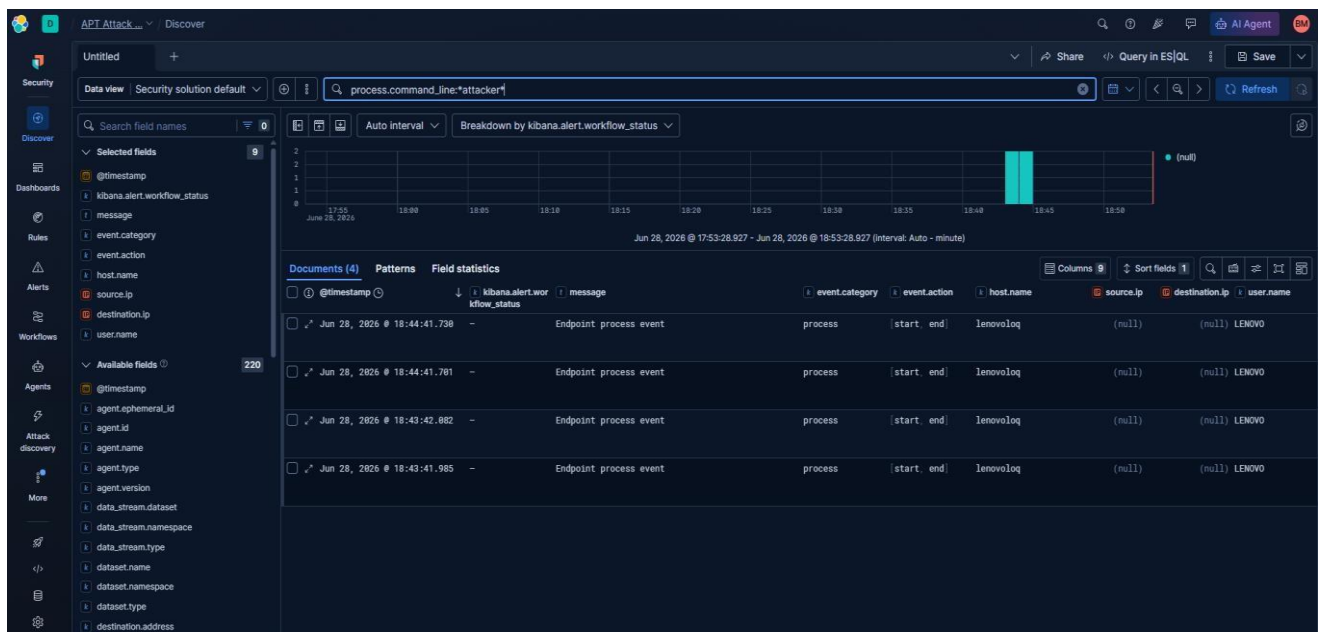
At the bottom, there is a 'Rows per page: 20' dropdown and pagination controls showing page 1 of 1.

Step 37 - Searching for Process Events

In this step, Kibana Discover was used to search for process events containing the term “attacker” in the command line. The search query `process.command_line:*attacker*` was entered.

The results displayed matching endpoint process events from the monitored system. This search was useful for checking whether any process or command containing the word “attacker” was recorded in the logs.

The event details, including the event category, event action, host name and user information, were visible in the results. This confirmed that the Elastic Agent was collecting and sending process activity to Kibana for further investigation.

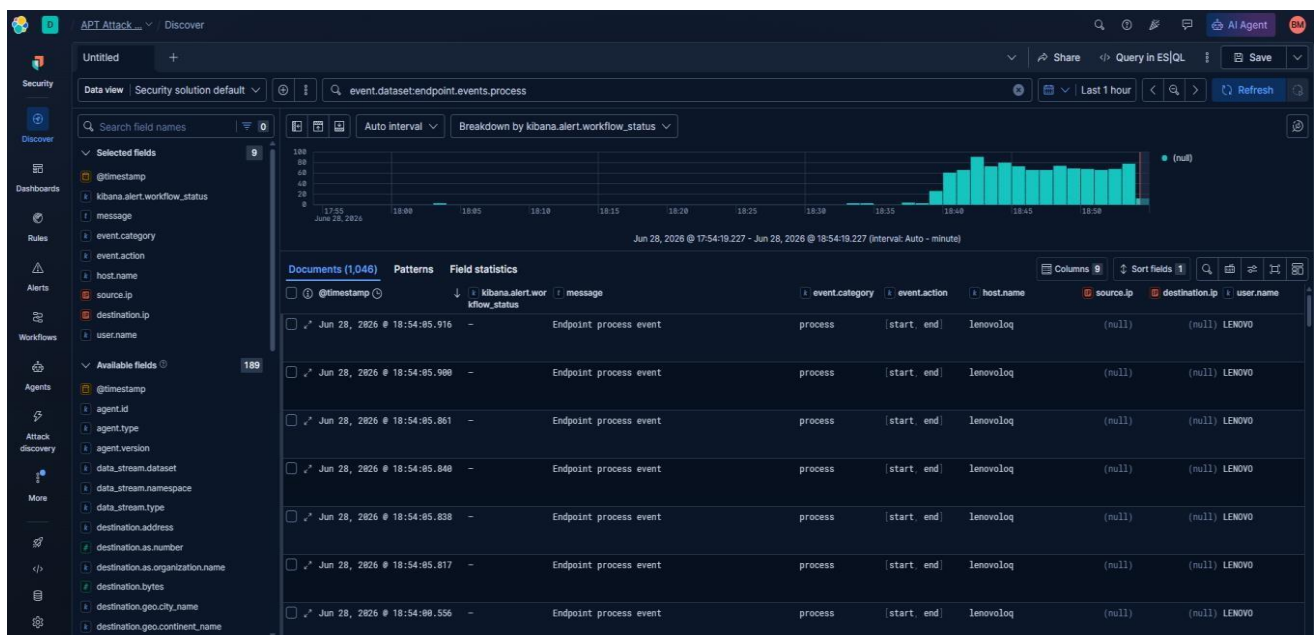


Step 38 - Viewing all Endpoint Process Events

In this step, the search query was changed to: **event.dataset:endpoint.events.process**

This query displayed the process-related events collected from the endpoint. A large number of events were available, showing that the Elastic Defend integration was actively monitoring process activity.

The results included information such as timestamps, event category, event action, host name and other related fields. Most of the displayed records were categorized as **process events**, showing that endpoint activity was successfully being recorded.

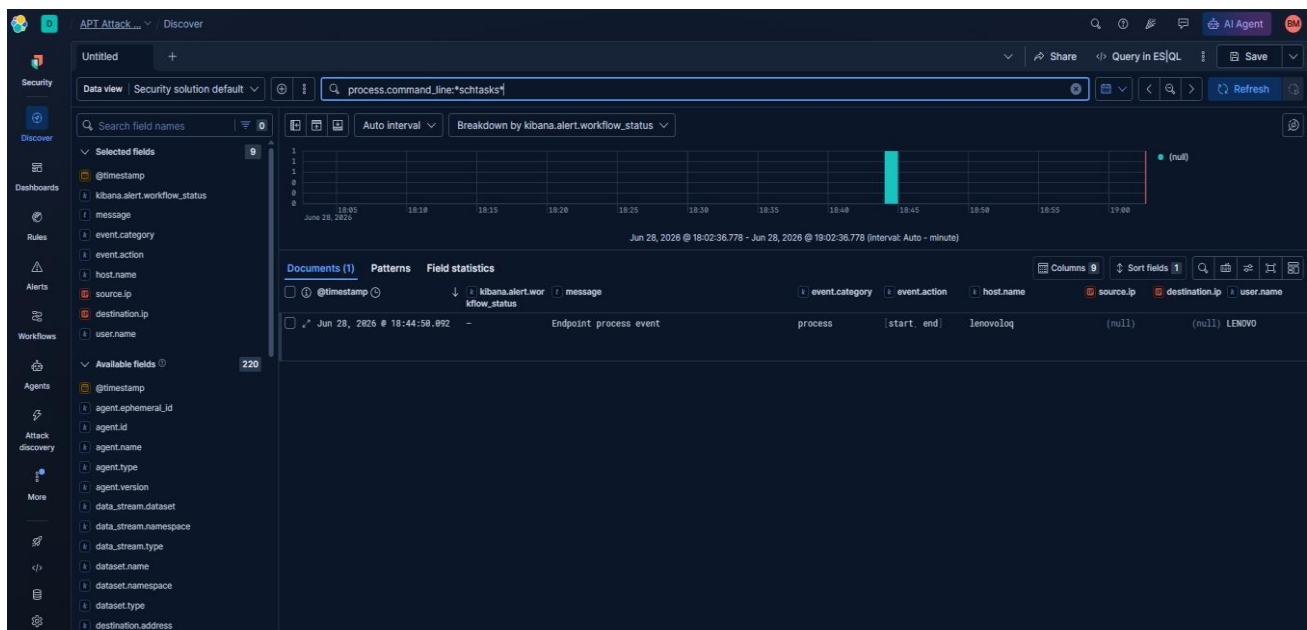


Step 39 - Searching for Scheduled Task Activity

In this step, the command line was searched using the query:
process.command_line:*schtasks*

The purpose of this search was to look for activity related to the Windows **schtasks** command.

The **schtasks** utility can be used to create, modify, or run scheduled tasks. Scheduled tasks are commonly used for legitimate administrative purposes, but they can also be used to maintain persistence on a system.



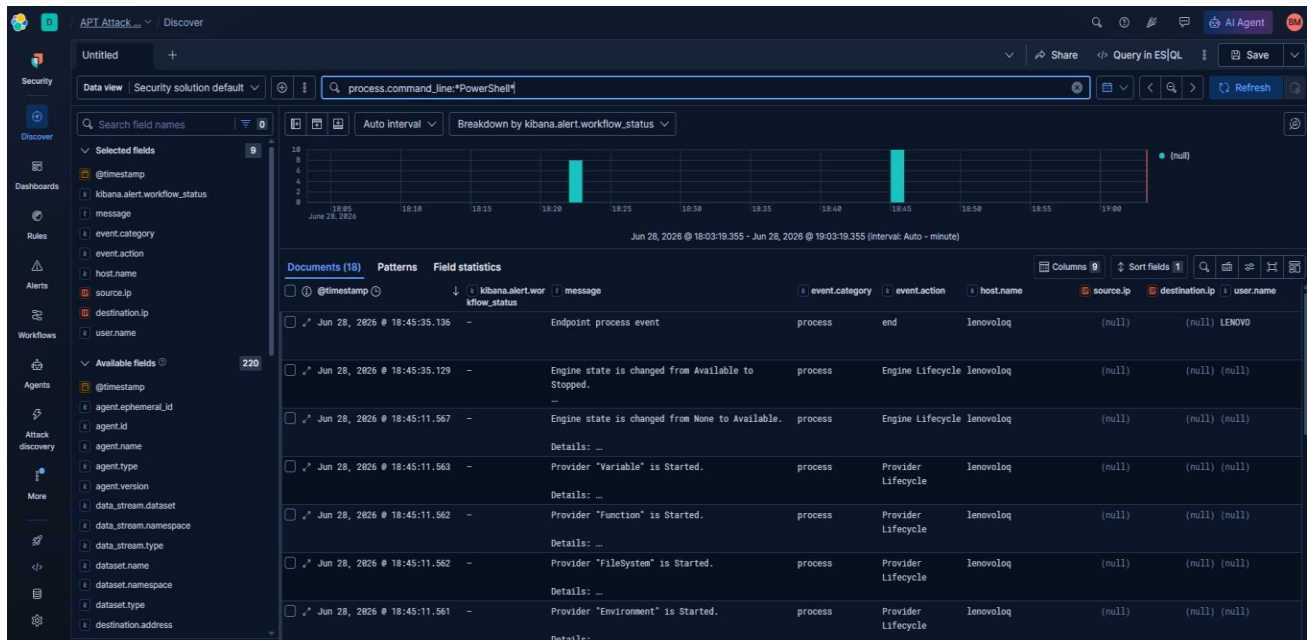
Step 40 - Searching for PowerShell Activity

In this step, the following search query was used:

process.command_line:*PowerShell*

The purpose was to identify process events related to PowerShell execution.

The search returned several events containing PowerShell-related activity. The displayed results included endpoint process events and other related system activity.



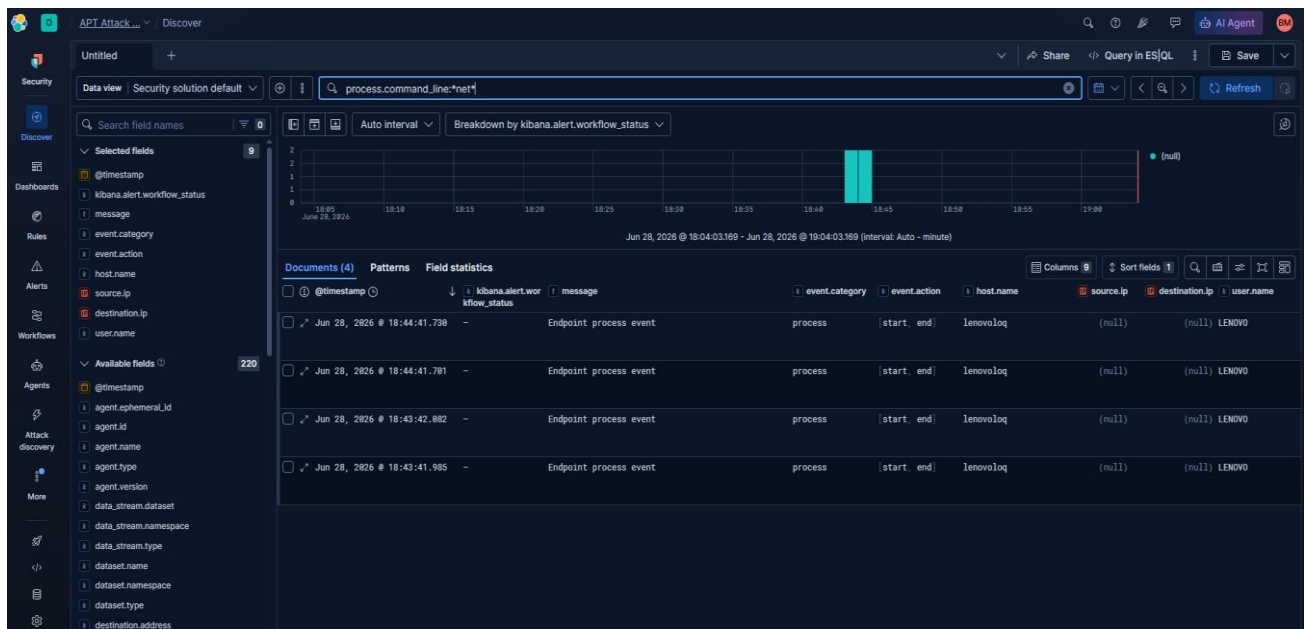
Step 41 - Searching for Network Command Activity

In this step, the command line was searched using:

process.command_line:*net*

The search was used to identify process command lines containing the term 'net', which may include activity related to the Windows net command.

The **net** command is commonly used in Windows to perform different administrative and network-related tasks. For example, it can be used to view users, groups, shared resources and other system information.



Step 42 - Searching for User-Related Commands

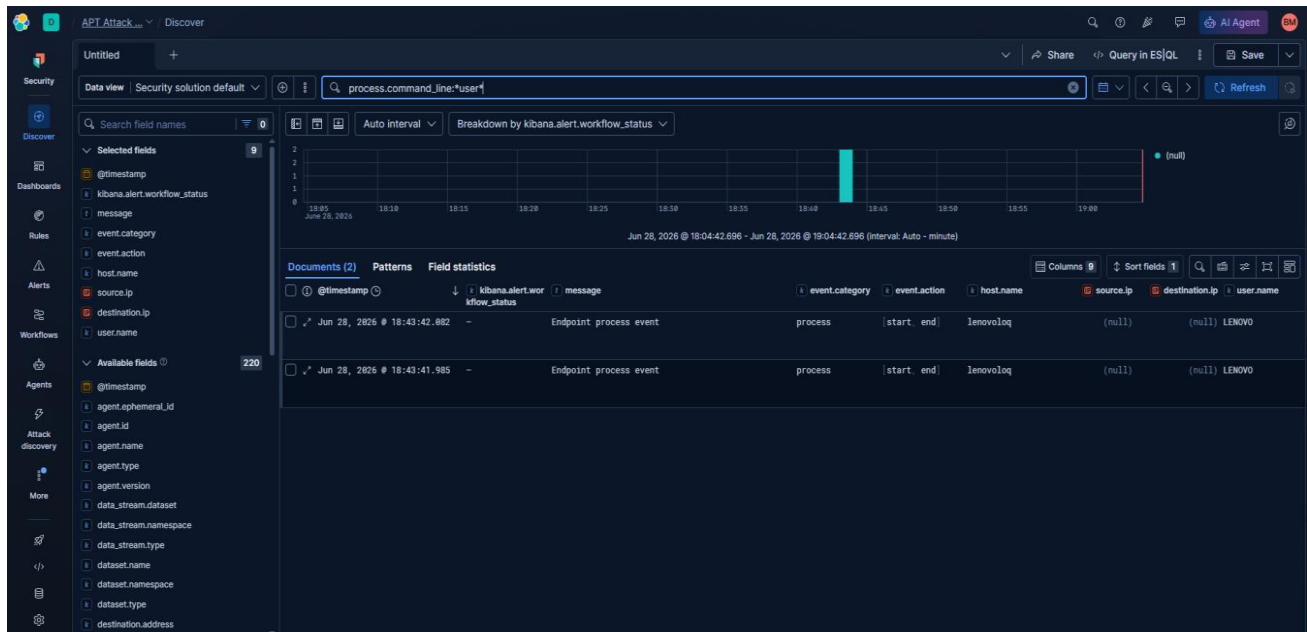
In this step, the following query was entered:

process.command_line:*user*

The purpose of this search was to identify process events containing the word **user**.

Commands containing this term may be related to checking user accounts, managing users, or collecting information about the users present on the system.

The results displayed matching process events from the monitored endpoint. By filtering the logs in this way, an analyst can focus on activities related to user accounts instead of manually checking every available event.



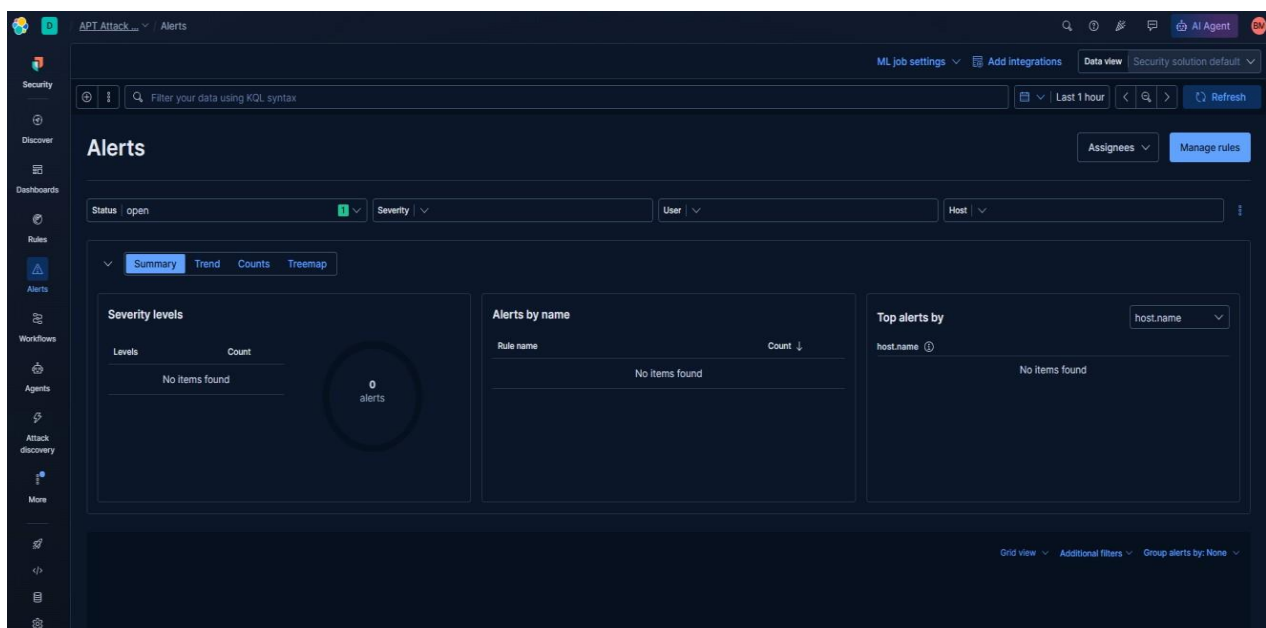
By performing searches for terms such as **attacker**, **schtasks**, **PowerShell**, **net** and **user**, it was possible to explore the endpoint logs in a more organized way. This approach can help a security analyst identify important commands and investigate potentially suspicious activity.

Step 43 - Viewing and Monitoring Security Alerts

After investigating the endpoint process activity, the next step was to open the **Alerts** section in Elastic Security. This page provides a central location for viewing and monitoring security alerts generated by detection rules and other security activities.

In this step, the Alerts page was opened with the **Status** filter set to **Open** and the time range set to **Last 1 hour**. The page displayed the alert summary, including sections for **Severity levels**, **Alerts by name** and **Top alerts by host**.

At the time of checking, the dashboard showed **0 alerts** and no items were found in the displayed sections. This means that no open security alerts were available during the selected time period.



5. Skills and Competencies Gained

Through this project, I gained practical knowledge and hands-on experience in different areas of cybersecurity, especially in **APT attack simulation, endpoint monitoring, security event analysis, and incident investigation.**

I learned how to set up and work with a small lab environment using **Kali Linux and Windows machines.** This helped me understand how an attacker machine and a target machine can communicate within a network. I also gained experience in checking network connectivity, identifying open ports and testing services using tools such as **Nmap.**

During the attack simulation, I understood how different attacker activities can be performed and observed from a security monitoring perspective. I practiced activities such as creating a test user, checking SMB and WinRM services, creating scheduled tasks for persistence, running PowerShell commands and collecting files to simulate data staging. These activities helped me understand how suspicious actions can leave traces on a system.

I also improved my knowledge of **Windows security monitoring.** By using Event Viewer, I learned how to check system events and verify activities such as scheduled task execution. This helped me understand the importance of logs during a security investigation.

I also gained knowledge about **detection rules and alerts.** I explored the available detection rules in Elastic Security and understood how these rules can help identify suspicious activities. I also checked the Alerts section to monitor whether any security alerts were generated during the testing period.

6. Feedback and Evidence

Feedback

This project gave me practical experience in simulating and investigating different security activities in a controlled lab environment. By using Kali Linux, Windows, and Elastic Security, I learned how attack-related activities can generate security events and how those events can be monitored and investigated.

During the project, I performed activities such as network scanning, service testing, creating a scheduled task for persistence, creating a test user, running PowerShell commands and simulating data collection. I also learned how to check Windows Event Viewer to verify system activities and understand the logs generated by these actions.

Evidence Collected

- Screenshots of network configuration and connectivity testing between the machines.
- Nmap scan screenshots showing the identified open ports and services on the Windows target machine.
- Screenshots showing SMB and WinRM service testing.
- Screenshots showing the creation of a test user and other simulated activities on the Windows machine.
- Screenshots showing the creation and execution of the scheduled task for persistence.
- Windows Event Viewer screenshots showing the events generated during the scheduled task activity.
- Screenshots showing the sample confidential file and simulated data collection/staging activity.
- Screenshots showing the Elastic Security environment and Windows integration setup.
- Screenshots showing the Elastic Agent installation, enrollment, and healthy agent status.
- Screenshots showing the Elastic Defend integration and Agent Policies.
- Screenshots showing the available and installed detection rules in Elastic Security.
- Kibana Discover screenshots showing endpoint process events and security log data.
- Screenshots showing searches for attacker-related activity, scheduled tasks, PowerShell, net commands, and user-related process activity.
- Screenshot of the Alerts page showing the current alert monitoring status during the selected time period.

7. Challenges and Solutions

During this project, I faced some challenges while setting up the lab environment and performing the APT attack simulation. Since I was still learning how different attack activities and security monitoring tools work, configuring the Kali Linux and Windows machines and making sure they could communicate properly required some time and practice. To overcome this, I checked the network settings and IP addresses and tested the connection between the systems before continuing with the remaining activities.

Testing services such as **SMB and WinRM** was also challenging because some of the connection attempts did not work as expected. I used Nmap to check the required ports and services and made changes to the Windows network and service settings. While testing remote access, I also faced permission and authorization issues. By checking the error messages and system configuration, I was able to understand how network settings, authentication and permissions can affect remote connections.

Another challenge was setting up **Elastic Security** and connecting the Windows system to it. The Windows integration, Elastic Agent, agent policy, and Elastic Defend integration had to be configured correctly for the endpoint events to be collected. To solve this, I completed the setup step by step and checked the agent status to confirm that the Windows system was successfully connected and reporting events.

Another challenge was that some test activities generated events but did not produce an alert during the selected time period. To understand this, I checked the collected events and the Alerts section separately. This helped me learn that an activity can be recorded in the logs even if it does not immediately create a security alert.

8. Outcomes and Impact

This project helped me gain practical experience in setting up a small cybersecurity lab and performing different activities related to an APT attack simulation. By using **Kali Linux, Windows, and Elastic Security**, I was able to understand how suspicious activities can be performed on a system and how the related events can be collected and investigated.

During the project, I successfully configured the Windows and Kali Linux machines and tested the communication between them. I used network scanning to identify available services and tested services such as **SMB and WinRM**. These activities helped me understand how a security analyst or attacker can identify services running on a target system.

9. Conclusion

This project gave me practical experience in setting up a small cybersecurity lab and performing different activities related to a multi-stage APT attack simulation. By using Kali Linux, Windows, and Elastic Security, I was able to understand how different activities can take place on a system and how the related events can be collected and investigated.

During the project, I set up the required lab environment and tested the communication between the Kali Linux and Windows machines. I used Nmap to identify open ports and services and performed testing on services such as SMB and WinRM. This helped me understand the importance of network services and how they can be checked during a security investigation.

Overall, this project improved my practical understanding of APT attack simulation, network service testing, Windows event monitoring, endpoint security, log analysis, and basic SOC investigation. It also helped me improve my troubleshooting and analytical skills by working through different configuration and monitoring challenges.

